

# Long-Context Stability and Hallucination Reduction in Large Language Models

March 2026

## Abstract

Large language models are increasingly judged by whether they can reliably use very long inputs and whether they can remain faithful to evidence instead of hallucinating. This note builds the topic from core concepts upward: tokenization, next-token prediction, self-attention, positional information, retrieval, verification, and citation. It then develops three formal views of the problem: a *long-context stability* view in which performance should not collapse as input length and answer position grow; a *faithful generation* view in which every answer claim should be supported by evidence; and a *systems* view in which retrieval, reranking, compression, verification, and citation are controllable modules.

Historically, the field progressed through several families of ideas: dense self-attention in the Transformer [1]; segment recurrence and relative positions in Transformer-XL [2]; compressed memory in Compressive Transformer [3]; efficient sparse attention in Reformer, Longformer, and BigBird [4, 5, 6]; extrapolatable positional schemes such as ALiBi and RoPE [11, 10]; and external-evidence systems such as REALM, RAG, FiD, RETRO, and Memorizing Transformers [7, 8, 9, 12, 13]. Modern work can be organized by two common frameworks. Framework A treats long-context extension as *position-geometry preservation*; it leads to methods such as Positional Interpolation (PI), YaRN, PoSE, LongLoRA, LongAlign, LongRoPE, and LongRoPE2 [19, 22, 23, 24, 30, 31, 40]. Framework B treats hallucination reduction as *evidence-grounded selective generation*; it leads to adaptive retrieval, critique, correction, graph-structured retrieval, and citation methods such as Self-RAG, CRAG, GraphRAG, LongCite, and SelfCite [32, 33, 34, 37, 42].

The main practical conclusion is that “longer context” and “retrieval augmentation” should not be treated as competing religions. They solve different failure modes. The most reliable systems are usually hybrid: stabilize the model’s use of long positions, retrieve and rerank only the most useful evidence, compress when necessary, and train or decode with explicit verification and citation objectives.

## How this note is organized

This note is organized around two recurring frameworks.

1. **Framework A: position-geometry preservation.** The goal is to keep long-context performance stable as input length and evidence position grow.
2. **Framework B: evidence-grounded selective generation.** The goal is to generate

answers whose claims can be traced to supporting evidence.

The main path starts with core concepts, then moves through historical roots, the two common frameworks, recent influential papers, and practical design rules.

**The central practical lesson**

Long-context extension and retrieval augmentation solve different failure modes. Robust systems usually combine *stable position handling inside the model* with *careful evidence control outside and around the model*.

Contents

|          |                                                                          |           |
|----------|--------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Reader’s guide: how this document is organized</b>                    | <b>5</b>  |
| 1.1      | A one-page map of the whole survey . . . . .                             | 5         |
| 1.2      | What you should know by page 10 . . . . .                                | 5         |
| 1.3      | A front-of-document glossary . . . . .                                   | 5         |
| 1.4      | One running example used everywhere . . . . .                            | 6         |
| <b>2</b> | <b>Core primer: from tokens to faithful long-context answers</b>         | <b>7</b>  |
| 2.1      | From raw text to tokens and embeddings . . . . .                         | 7         |
| 2.2      | What the model predicts: next-token prediction . . . . .                 | 8         |
| 2.3      | Training versus inference: teacher forcing and the causal mask . . . . . | 9         |
| 2.4      | Why attention exists at all . . . . .                                    | 10        |
| 2.5      | Multi-head attention and the full transformer block . . . . .            | 11        |
| 2.6      | Why positions matter . . . . .                                           | 12        |
| 2.7      | What retrieval does . . . . .                                            | 13        |
| 2.8      | What hallucination means, and what it does not mean . . . . .            | 14        |
| 2.9      | Why citations matter . . . . .                                           | 15        |
| 2.10     | Checkpoint before moving on . . . . .                                    | 15        |
| <b>3</b> | <b>Formal problem setup and evaluation lenses</b>                        | <b>16</b> |
| 3.1      | Problem 1: long-context stability . . . . .                              | 16        |
| 3.2      | Problem 2: hallucination reduction and claim support . . . . .           | 17        |
| 3.3      | Three places where systems can fail . . . . .                            | 17        |
| 3.4      | What the main benchmarks actually measure . . . . .                      | 18        |

---

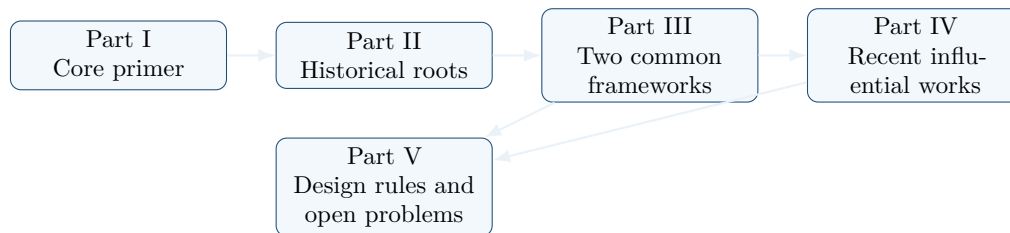
|          |                                                                                      |           |
|----------|--------------------------------------------------------------------------------------|-----------|
| <b>4</b> | <b>Historical roots, organized by family rather than by year</b>                     | <b>18</b> |
| 4.1      | A timeline before the details . . . . .                                              | 18        |
| 4.2      | Family 1: the dense-attention root — the Transformer . . . . .                       | 19        |
| 4.3      | Family 2: extending memory beyond one segment . . . . .                              | 20        |
| 4.4      | Family 3: reducing the quadratic cost of attention . . . . .                         | 23        |
| 4.5      | Family 4: position handling and extrapolation . . . . .                              | 25        |
| 4.6      | Family 5: external evidence and retrieval as memory . . . . .                        | 27        |
| 4.7      | Historical synthesis: what the roots gave us . . . . .                               | 30        |
| <b>5</b> | <b>Common Framework A: position-geometry preservation for long-context stability</b> | <b>30</b> |
| 5.1      | Why RoPE-based models often break when context becomes very long . . . . .           | 30        |
| 5.2      | A generic remapping view . . . . .                                                   | 31        |
| 5.3      | Positional Interpolation (PI): the canonical first modern recipe . . . . .           | 31        |
| 5.4      | YaRN: interpolation plus targeted correction . . . . .                               | 33        |
| 5.5      | PoSE, LongLoRA, and LongAlign: make adaptation cheaper . . . . .                     | 33        |
| 5.6      | LongRoPE and LongRoPE2: nonuniform search and progressive extension . . . . .        | 35        |
| 5.7      | Complementary ideas: StreamingLLM, Infini-attention, and ReAttention . . . . .       | 36        |
| 5.8      | A unified derivation of the final algorithmic template for Framework A . . . . .     | 36        |
| 5.9      | How the community learned to evaluate Framework A . . . . .                          | 37        |
| <b>6</b> | <b>Common Framework B: evidence-grounded selective generation</b>                    | <b>38</b> |
| 6.1      | The end-to-end problem in plain language . . . . .                                   | 38        |
| 6.2      | Latent-evidence formulation . . . . .                                                | 39        |
| 6.3      | Retrieval, reranking, and compression as optimization subproblems . . . . .          | 39        |
| 6.4      | From evidence selection to faithful answers . . . . .                                | 40        |
| 6.5      | Verification and revision: RARR, CoVe, and SelfCheckGPT . . . . .                    | 41        |
| 6.6      | Adaptive retrieval and correction: Self-RAG and CRAG . . . . .                       | 41        |
| 6.7      | Structured retrieval: GraphRAG, LongRAG, and R <sup>2</sup> AG . . . . .             | 42        |
| 6.8      | Citation as a first-class objective: LongCite and SelfCite . . . . .                 | 43        |
| 6.9      | A unified derivation of the final algorithmic template for Framework B . . . . .     | 44        |
| 6.10     | How the community evaluates Framework B . . . . .                                    | 45        |
| <b>7</b> | <b>Recent influential works (2023–2025), separated into canon and frontier</b>       | <b>45</b> |
| 7.1      | Main-text canon: the papers a newcomer should know first . . . . .                   | 45        |

---

|          |                                                                            |           |
|----------|----------------------------------------------------------------------------|-----------|
| 7.2      | How each canonical paper fits the two-framework view . . . . .             | 46        |
| 7.3      | Promising frontier papers whose final status is still maturing . . . . .   | 50        |
| 7.4      | A practical lesson from the recent literature . . . . .                    | 50        |
| <b>8</b> | <b>Design rules, failure-analysis checklist, and open problems</b>         | <b>51</b> |
| 8.1      | When should you use a larger context window, retrieval, or both? . . . . . | 51        |
| 8.2      | A failure-analysis checklist for practitioners . . . . .                   | 51        |
| 8.3      | What remains genuinely hard . . . . .                                      | 52        |
| 8.4      | If you only remember six ideas from the whole survey . . . . .             | 52        |
| <b>9</b> | <b>Conclusion</b>                                                          | <b>52</b> |
| <b>A</b> | <b>Optional reading path for an even slower first pass</b>                 | <b>53</b> |
| <b>B</b> | <b>Notation summary</b>                                                    | <b>53</b> |

# 1 Reader's guide: how this document is organized

## 1.1 A one-page map of the whole survey



**Figure 1:** Dependency structure for the note. A first pass reads left to right; experienced readers can jump to the frameworks or recent-work sections.

### Plain-language summary

**How to use the three layers.** Each major topic is presented in three layers.

1. **Layer 1: intuition.** Plain English and toy examples.
2. **Layer 2: core mechanism.** One or two equations and the central algorithmic idea.
3. **Layer 3: derivation.** Step-by-step mathematical development from the problem to the final algorithmic form.

On a first pass, you can read only Layers 1 and 2 and still understand the field at a meaningful level. On a second pass, read Layer 3 carefully.

## 1.2 What you should know by page 10

By the end of the core primer, you should be able to say, in your own words:

1. what a token is,
2. what the model predicts,
3. why attention uses queries, keys, and values,
4. why position information is necessary,
5. why long contexts can fail even when the answer is present,
6. what retrieval adds beyond a large context window,
7. why hallucination is not the same as retrieval failure,
8. why citation is a stronger requirement than just giving a plausible answer.

## 1.3 A front-of-document glossary

| Term                   | Plain meaning                                                                                                                                | You first need it in |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|----------------------|
| Token                  | A basic symbol unit seen by the model. A token can be a word, part of a word, punctuation, or a special marker.                              | Section 2            |
| Embedding              | A learned vector representation of a token.                                                                                                  | Section 2            |
| Logit                  | A raw score before probabilities are formed by softmax.                                                                                      | Section 2            |
| Causal mask            | A rule that prevents token $t$ from looking at future tokens $> t$ .                                                                         | Section 2            |
| Query / key / value    | Three learned projections used by attention to decide <i>what to look for</i> , <i>where to look</i> , and <i>what information to copy</i> . | Section 2            |
| Context window         | The maximum number of tokens a model can process at once.                                                                                    | Section 3            |
| RoPE                   | Rotary position embedding; it injects position by rotating pairs of coordinates.                                                             | Section 4            |
| Retrieval              | Selecting external passages or documents that may contain useful evidence.                                                                   | Section 2            |
| Reranking              | Reordering retrieved candidates using a stronger but more expensive relevance model.                                                         | Section 2            |
| Hallucination          | A generated statement that is unsupported, false, or contradicted by the evidence.                                                           | Section 2            |
| Citation               | A link from a generated claim to the supporting evidence span.                                                                               | Section 2            |
| Long-context stability | The property that performance should remain strong as inputs and answer locations become much longer than in training.                       | Section 3            |

## 1.4 One running example used everywhere

Throughout the survey, keep one task in mind.

### Plain-language summary

A user asks: “According to a 300-page policy manual and its appendices, which rules govern incident reporting, who signs off, and which exceptions apply? Please provide sentence-level citations.”

This single task lets us see several different failure modes.

- If the correct paragraph is on page 180 and the model ignores it, that is a **long-context stability failure**.

- If the answer sounds confident but is not supported by the manual, that is a **hallucination**.
- If the answer is right but the citation points to the wrong sentence, that is a **citation failure**.
- If retrieval misses the relevant appendix entirely, that is a **retrieval failure**.

## 2 Core primer: from tokens to faithful long-context answers

### Plain-language summary

This section establishes the core mental model for the rest of the note. It does not assume prior knowledge of transformers. If a later section feels too abstract, come back here.

### 2.1 From raw text to tokens and embeddings

#### Layer 1: intuition

A language model does not directly read a sentence the way a human does. It first breaks the sentence into smaller units called *tokens*. Then each token is mapped to a vector, called an *embedding*. Once text becomes vectors, the model can apply linear algebra to it.

Consider the sentence

The manager signs the incident report.

A tokenizer might split it into tokens such as

[The] [manager] [signs] [the] [incident] [report] [..].

In other vocabularies it might split **incident** or **report** into smaller pieces. The exact split is not the important part. The important part is that the model sees a sequence of token IDs.

#### Layer 2: core mechanism

Let the vocabulary size be  $|V|$  and the hidden dimension be  $d$ . The embedding matrix is

$$E \in \mathbb{R}^{|V| \times d}. \quad (2.1)$$

If token  $w_t$  is the  $t$ th token ID, its embedding is the row vector

$$x_t = E[w_t] \in \mathbb{R}^d. \quad (2.2)$$

*Meaning.* The model begins with a list of vectors  $x_1, \dots, x_n$  rather than a list of words.

#### Common confusion

**Tokens are not the same as words.** This matters when we talk about context windows. A 100-page document may contain far more than 100 pages' worth of tokens because punctuation,

subword pieces, tables, and code can all increase token count.

### What to remember

Text becomes token IDs. Token IDs become embeddings. From then on, the model only manipulates vectors.

## 2.2 What the model predicts: next-token prediction

### Layer 1: intuition

A causal language model is trained to do one simple thing over and over: look at the tokens so far and guess the next token. Even very complex behavior emerges from repeating this simple objective at scale.

Suppose the prefix is

The manager signs the incident

and the true next token is **report**. The model produces a score for every vocabulary token. These raw scores are called *logits*. A softmax then turns them into probabilities.

### Layer 2: core mechanism

Let  $z_t \in \mathbb{R}^{|V|}$  be the logit vector at position  $t$ . The predicted distribution is

$$p_\theta(w_t = v \mid w_{<t}) = \frac{\exp(z_{t,v})}{\sum_{v' \in V} \exp(z_{t,v'})}. \quad (2.3)$$

Over a whole sequence  $w_{1:n}$ , the model factorizes the joint probability as

$$p_\theta(w_{1:n}) = \prod_{t=1}^n p_\theta(w_t \mid w_{<t}). \quad (2.4)$$

Training minimizes negative log-likelihood:

$$\mathcal{L}_{\text{LM}}(\theta) = -\mathbb{E}_{w_{1:n} \sim \mathcal{D}} \sum_{t=1}^n \log p_\theta(w_t \mid w_{<t}). \quad (2.5)$$

*Meaning.* The model is rewarded when it assigns high probability to the true next token at every position.

### Layer 3: derivation

Why does Eq. (2.4) make sense? Start from the chain rule of probability:

$$p(w_{1:n}) = p(w_1) p(w_2 \mid w_1) \cdots p(w_n \mid w_{1:n-1}). \quad (2.6)$$

A causal language model simply parameterizes each conditional factor with one shared neural

network. The cross-entropy loss in Eq. (2.5) is the negative log of that chain-rule factorization. There is no extra magic hidden here.

### Common confusion

**Training and generation are not identical.** During training, the model sees the true previous token. During generation, it sees its own previously sampled token. This gap is one reason fluent error chains can happen.

## 2.3 Training versus inference: teacher forcing and the causal mask

### Layer 1: intuition

During training, the model predicts every next token in parallel because the full target sentence is already known. During generation, it must produce one token, append it, then continue. This is why generation is sequential even though training can be highly parallel.

### Layer 2: core mechanism

The model is trained with *teacher forcing*: at position  $t$ , it conditions on the true prefix  $w_{<t}$ . To prevent information leakage from the future, self-attention uses a *causal mask*

$$M_{ij} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i. \end{cases} \quad (2.7)$$

Adding  $M$  to the attention scores ensures that token  $i$  cannot attend to future tokens.

### Algorithmic template

#### Autoregressive generation.

1. Start from a prompt  $w_{1:t_0}$ .
2. Compute the next-token distribution  $p_{\theta}(w_{t_0+1} \mid w_{1:t_0})$ .
3. Choose the next token by greedy decoding, sampling, or beam search.
4. Append that token to the context.
5. Repeat until an end condition is reached.

### What to remember

The model learns with the true past but generates using its own past outputs. Causal masking keeps the task honest.

## 2.4 Why attention exists at all

### Layer 1: intuition

Suppose the current token is **signs**. To predict what comes next, the model should care more about **manager** and **incident report** than about a distant punctuation mark. A fixed average of all previous tokens would be too crude. Attention learns a *data-dependent weighted average*. Different tokens can look at different parts of the prefix.

Before we write the full equation, start with a simpler idea. Let  $v_1, \dots, v_i$  be candidate information vectors available before position  $i$ . If we choose nonnegative weights  $\alpha_{ij}$  that sum to one, then

$$\hat{h}_i = \sum_{j \leq i} \alpha_{ij} v_j \quad (2.8)$$

is a weighted average of past information. Attention is the learned rule that chooses these weights.

### Layer 2: core mechanism

For one attention head, the model forms three learned projections of each hidden state  $h_t$ :

$$q_t = W_Q h_t, \quad k_t = W_K h_t, \quad v_t = W_V h_t. \quad (2.9)$$

These are called the *query*, *key*, and *value*. The score from position  $i$  to position  $j$  is

$$s_{ij} = \frac{q_i^\top k_j}{\sqrt{d_k}} + M_{ij}. \quad (2.10)$$

The attention weights are

$$\alpha_{ij} = \frac{\exp(s_{ij})}{\sum_{m \leq i} \exp(s_{im})}, \quad (2.11)$$

and the output is

$$\hat{h}_i = \sum_{j \leq i} \alpha_{ij} v_j. \quad (2.12)$$

*Meaning.* The query asks “what am I looking for?”; keys say “what kind of information do I contain?”; values are the information that gets copied once relevance is decided.

### Layer 3: derivation

Why use dot products and softmax? There are three practical requirements.

1. We need a similarity function between the current need  $q_i$  and candidate content  $k_j$ . A dot product is the simplest learned bilinear score.
2. We need comparable scales across dimensions. Dividing by  $\sqrt{d_k}$  prevents scores from growing too large when  $d_k$  is large.
3. We need a nonnegative, normalized weight vector. The softmax converts arbitrary real-valued scores into weights on the probability simplex, guaranteeing  $\alpha_{ij} \geq 0$  and  $\sum_j \alpha_{ij} = 1$ .

Thus Eqs. (2.10)–(2.12) are a natural learned version of similarity-weighted averaging.

### 2.4.1 A tiny numerical example

Suppose the current token is **signs** and the model is choosing how much to look at three earlier tokens: **manager**, **incident**, and **the**. Imagine the attention scores are

$$(s_{i,\text{manager}}, s_{i,\text{incident}}, s_{i,\text{the}}) = (2.0, 1.0, 0.0). \quad (2.13)$$

Then softmax gives approximately

$$(\alpha_{i,\text{manager}}, \alpha_{i,\text{incident}}, \alpha_{i,\text{the}}) \approx (0.665, 0.245, 0.090). \quad (2.14)$$

So the model copies much more information from **manager** than from **the**. That is exactly the kind of selective focus we want.

#### Common confusion

**Attention is not explanation by itself.** High attention on a token can be informative, but it is not a complete proof of causality or reasoning. Later citation methods aim for a stronger notion of support.

## 2.5 Multi-head attention and the full transformer block

### Layer 1: intuition

One attention head may specialize in one kind of relationship, such as subject–verb agreement, coreference, or retrieving a relevant instruction. Multiple heads let the model look for several kinds of patterns in parallel.

### Layer 2: core mechanism

For head  $m = 1, \dots, H$ , compute

$$\text{head}^{(m)} = \text{Attn}(HW_Q^{(m)}, HW_K^{(m)}, HW_V^{(m)}), \quad (2.15)$$

where  $H \in \mathbb{R}^{L \times d}$  stacks the hidden states of all  $L$  positions. Concatenate the head outputs and project:

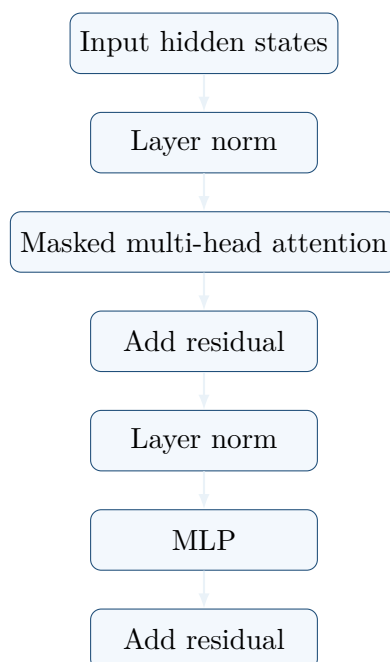
$$\text{MHA}(H) = \text{Concat}(\text{head}^{(1)}, \dots, \text{head}^{(H)})W_O. \quad (2.16)$$

A standard decoder block then applies

$$\tilde{H} = H + \text{MHA}(\text{LN}(H)), \quad (2.17)$$

$$H' = \tilde{H} + \text{MLP}(\text{LN}(\tilde{H})). \quad (2.18)$$

*Meaning.* Residual connections keep old information available while each sublayer adds a useful correction.



**Figure 2:** A minimal view of one decoder-only transformer block.

### What to remember

A transformer stack alternates between mixing information *across positions* (attention) and transforming information *within each position* (MLP).

## 2.6 Why positions matter

### Layer 1: intuition

If a model only saw token embeddings and no positional information, it could not tell the difference between “dog bites man” and “man bites dog” if the multiset of tokens were the same. Order matters, so position must enter the model somehow.

### Layer 2: core mechanism

There are two broad ways to add positional information.

1. **Add position vectors to token states.** Then the hidden state at position  $i$  is  $h_i = x_i + p_i$ .
2. **Modify attention scores directly.** Relative-position methods such as ALiBi and RoPE change the score between positions  $i$  and  $j$  so it depends on their distance or relative phase.

**Common confusion**

**A long context window is not the same as long-context skill.** A model may accept 128k tokens and still use them poorly. The question is not only *can it fit the tokens?* but also *can it find and use the right ones?*

**2.7 What retrieval does****Layer 1: intuition**

A long context window means the model can read more text at once. Retrieval means we do not ask it to read everything. Instead, we search a larger external collection and bring back only the most relevant evidence. These are different tools.

Let the user’s question be  $x$  and the external corpus be  $\mathcal{D} = \{d_1, \dots, d_M\}$ . A retrieval system usually has three stages.

1. **Initial retrieval:** find candidate passages quickly.
2. **Reranking:** reorder those candidates using a stronger relevance model.
3. **Compression / packing:** fit the selected evidence into the prompt budget.

**Layer 2: core mechanism**

A retrieval score function is simply a function  $r(x, d)$  that estimates how relevant document  $d$  is to query  $x$ .

- In lexical retrieval,  $r$  depends on word overlap, inverse document frequency, and field statistics.
- In dense retrieval, the query and document are embedded into vectors  $u(x)$  and  $v(d)$ , and a common score is

$$r(x, d) = u(x)^\top v(d). \quad (2.19)$$

The retriever returns the top- $k$  candidates

$$\text{top-}k_{d \in \mathcal{D}} r(x, d). \quad (2.20)$$

**2.7.1 A toy retrieval example**

Suppose the question is “*Who signs off incident reports?*” and the corpus contains five passages.

| ID    | Passage summary                                          | Initial score |
|-------|----------------------------------------------------------|---------------|
| $d_1$ | General policy overview                                  | 0.20          |
| $d_2$ | Incident reporting workflow; says the line manager signs | 0.91          |
| $d_3$ | Vacation approval workflow                               | 0.44          |
| $d_4$ | Historical policy no longer in force                     | 0.53          |
| $d_5$ | Exception rules for emergency cases                      | 0.77          |

The top initial hits are  $d_2, d_5, d_4$ . A reranker may then demote  $d_4$  because it is obsolete and keep  $d_2, d_5$ . Those two passages are what we pass into the language model.

### What to remember

Retrieval is a selection tool. It tries to reduce a huge search space down to a small set of promising evidence passages.

## 2.8 What hallucination means, and what it does not mean

### Layer 1: intuition

A hallucination is not merely “an answer I dislike.” It is a mismatch between the generated statement and the evidence or the world. In grounded question answering, the cleanest definition is evidence-based: a claim hallucinates if it is unsupported or contradicted by the provided sources.

We will use four recurring error types.

1. **Unsupported insertion:** the answer adds a detail not present in the evidence.
2. **Contradiction:** the answer conflicts with the evidence.
3. **Conflation:** the answer mixes facts from multiple sources into an incorrect composite.
4. **Citation error:** the answer text may be plausible, but the cited span does not actually support it.

### Layer 2: core mechanism

If an answer  $y$  contains claim sentences  $c_1, \dots, c_m$  and the selected evidence is  $E$ , we can define a support indicator

$$\text{supp}(c_j, E) = \begin{cases} 1, & \text{if } E \text{ entails or directly supports claim } c_j, \\ 0, & \text{otherwise.} \end{cases} \quad (2.21)$$

A simple claim-level faithfulness score is then

$$F(y, E) = \frac{1}{m} \sum_{j=1}^m \text{supp}(c_j, E). \quad (2.22)$$

*Meaning.* Faithfulness is higher when more answer claims are directly supported by evidence.

### Common confusion

**Retrieval failure and hallucination are different.** If the right evidence never reaches the model, the model may still hallucinate, but the upstream cause is retrieval failure. Good systems separate these error sources.

## 2.9 Why citations matter

### Layer 1: intuition

A correct answer is good. A correct answer with a precise supporting sentence is much better because a human can verify it quickly. Citation is therefore a trust and usability feature, not only an academic nicety.

### Layer 2: core mechanism

If claim  $c_j$  is supported by a subset of evidence spans  $\mathcal{S}_j$ , a citation system aims to output one or more spans  $\hat{\mathcal{S}}_j$  such that

necessity: removing  $\hat{\mathcal{S}}_j$  hurts the claim,      sufficiency: keeping only  $\hat{\mathcal{S}}_j$  preserves the claim. (2.23)

This necessity–sufficiency idea will reappear later in SelfCite-style objectives.

## 2.10 Checkpoint before moving on

### Quick self-test

You are ready for the rest of the survey if you can answer “yes” to most of these.

1. I know that the model predicts the next token, not an entire paragraph in one step.
2. I know that attention is a learned weighted average over previous tokens.
3. I know why a causal mask is necessary.
4. I know that position information must be injected somehow.
5. I know that a long context window and long-context skill are not the same thing.
6. I know that retrieval selects external evidence instead of forcing the model to read everything.
7. I know that hallucination means lack of support or contradiction, not just low style quality.

### 3 Formal problem setup and evaluation lenses

#### Plain-language summary

The core primer explained the ideas informally. This section turns those ideas into precise problem statements. The goal is not to make the topic harder; it is to make the later derivations cleaner.

#### 3.1 Problem 1: long-context stability

##### Layer 1: intuition

A model is *long-context stable* if its performance does not collapse merely because the document became much longer or because the relevant evidence moved deeper into the document.

Let  $x_{1:\ell}$  denote an input of length  $\ell$ , and let  $y$  be the desired output. We will often care not only about the total input length  $\ell$  but also about the position  $\pi$  of the answer-bearing evidence inside the input.

##### Layer 2: core mechanism

Let  $\mathcal{M}(f; \ell, \pi)$  be a performance metric for model  $f$  on examples of length  $\ell$  whose decisive evidence lies around position  $\pi$ . A useful stability gap is

$$\Delta_{\text{LC}}(f; \ell, \pi) = \mathcal{M}(f; \ell_0, \pi_0) - \mathcal{M}(f; \ell, \pi), \quad (3.1)$$

where  $(\ell_0, \pi_0)$  is an easy baseline regime, such as short contexts with the answer near the end. A model is more stable when this degradation remains small even as  $\ell$  and  $\pi$  increase.

A more probabilistic view is to define a task distribution  $\mathcal{T}$  over tuples  $(x, y, \ell, \pi)$  and minimize expected loss

$$R_{\text{LC}}(f) = \mathbb{E}_{(x, y, \ell, \pi) \sim \mathcal{T}} [\mathcal{L}(f(x_{1:\ell}), y)]. \quad (3.2)$$

Long-context research is essentially about reducing this risk in the regime where  $\ell$  becomes much larger than the positions seen during training.

##### Common confusion

The community sometimes advertises a model's *maximum accepted context length*. That is an engineering property. Stability asks a stronger question: does the model still use the context well?

### 3.2 Problem 2: hallucination reduction and claim support

#### Layer 1: intuition

In grounded generation, we do not only care whether the final answer looks plausible. We care whether each claim in the answer is supported by evidence.

Suppose a generated answer  $y$  is decomposed into atomic claims  $\mathcal{C}(y) = \{c_1, \dots, c_m\}$  and the evidence available to the model is  $E$ .

#### Layer 2: core mechanism

Define a claim-level unsupported-risk objective:

$$R_{\text{hall}}(f) = \mathbb{E}_{(x,E)} \left[ \frac{1}{|\mathcal{C}(y)|} \sum_{c \in \mathcal{C}(y)} (1 - \text{supp}(c, E)) \right], \quad y = f(x, E). \quad (3.3)$$

Lower is better. If a claim is unsupported or contradicted, it contributes error.

When citations are required, we need an additional span-selection function. Let  $z(c)$  denote the evidence span selected as support for claim  $c$ .

$$R_{\text{cite}}(f) = \mathbb{E}_{(x,E)} \left[ \frac{1}{|\mathcal{C}(y)|} \sum_{c \in \mathcal{C}(y)} (1 - \text{supp}(c, z(c))) \right]. \quad (3.4)$$

This risk is harder because it requires not just a truthful claim, but a truthful claim with the right supporting span.

### 3.3 Three places where systems can fail

Both long-context and grounded-generation systems have multiple failure points.

1. **Inside-model failure:** the transformer itself mishandles positions, attention allocation, or long-range memory.
2. **Evidence-selection failure:** retrieval or reranking misses the right passage or includes too much distractor text.
3. **Answering failure:** the model sees the evidence but still summarizes it incorrectly, overgeneralizes, or cites it badly.

This decomposition matters because different techniques target different failure points.

- PI, YaRN, LongRoPE, and related methods mainly target inside-model position failures.
- RAG, FiD, and GraphRAG mainly target evidence-selection and evidence-presentation failures.
- Self-RAG, CRAG, CoVe, LongCite, and SelfCite mainly target answering, verification, and attribution failures.

### 3.4 What the main benchmarks actually measure

| Benchmark family                        | Main question                                                                                                | What it is useful for                                                                    |
|-----------------------------------------|--------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Needle-style tests                      | Can the model locate a planted fact in long distractor text?                                                 | Good for basic retrieval-from-context checks, but too shallow to measure deep reasoning. |
| LongBench / LongBench v2                | Can the model handle realistic long documents across QA, summarization, dialogue, code, and structured data? | More realistic long-context understanding; LongBench v2 increases reasoning difficulty.  |
| RULER / $\infty$ Bench                  | Does performance hold as sequence length and task complexity increase?                                       | Stress tests for effective, not just nominal, context size.                              |
| Truthfulness / hallucination benchmarks | Does the model avoid unsupported or false claims?                                                            | Useful when we care about factuality beyond retrieval.                                   |
| Citation benchmarks                     | Can the model attach precise supporting spans to answer sentences?                                           | Needed when verifiability matters, not only answer correctness.                          |

**Table 2:** Different benchmark families answer different questions. No single benchmark settles the entire long-context + hallucination problem.

#### What to remember

The field is easier to understand once you separate *inside-model long-context stability* from *outside-model evidence control*. Many papers sit mostly on one side or the other.

## 4 Historical roots, organized by family rather than by year

### Plain-language summary

The historical literature can feel confusing because many papers attack different bottlenecks at once. This section groups the roots by the bottleneck they target: dense attention, memory extension, cost reduction, positional extrapolation, and external evidence.

### 4.1 A timeline before the details

| Year | Work            | Family               | Why it mattered                                    |
|------|-----------------|----------------------|----------------------------------------------------|
| 2017 | Transformer [1] | Dense attention root | Made full self-attention the central architecture. |

| Year | Work                         | Family                     | Why it mattered                                                                          |
|------|------------------------------|----------------------------|------------------------------------------------------------------------------------------|
| 2019 | Transformer-XL [2]           | Memory extension           | Reused hidden states across segments; introduced strong relative-position decomposition. |
| 2020 | Compressive Trans-former [3] | Memory extension           | Added compressed long-term memory beyond Transformer-XL.                                 |
| 2020 | Reformer [4]                 | Cost reduction             | Approximated attention with locality-sensitive hashing.                                  |
| 2020 | Longformer [5]               | Cost reduction             | Used sparse sliding-window + global attention for long documents.                        |
| 2020 | BigBird [6]                  | Cost reduction             | Used local + global + random sparse patterns with stronger theory.                       |
| 2020 | REALM [7]                    | External evidence          | Made retrieved documents latent variables during pretraining.                            |
| 2020 | RAG [8]                      | External evidence          | Canonical retrieval-augmented generation formulation.                                    |
| 2021 | FiD [9]                      | External evidence          | Strong passage fusion architecture for open-domain QA.                                   |
| 2021 | RoPE [10]                    | Position geometry          | Relative position via coordinate rotations; foundational for later long-context work.    |
| 2022 | ALiBi [11]                   | Position geometry          | Simple linear bias with good extrapolation behavior.                                     |
| 2022 | RETRO [12]                   | External evidence          | Retrieval inserted inside the LM loop at chunk level.                                    |
| 2022 | Memorizing Transformers [13] | External evidence / memory | Added nearest-neighbor external memory directly to attention.                            |

## 4.2 Family 1: the dense-attention root — the Transformer

### 4.2.1 Transformer (Vaswani et al., 2017)

#### Layer 1: intuition

The original Transformer solved a major bottleneck of recurrent models: instead of processing tokens strictly one by one and passing a single state forward, it let every token directly look at every other relevant token through self-attention.

#### Layer 2: core mechanism

In matrix form, one attention head is

$$\text{Attn}(Q, K, V) = \text{softmax} \left( \frac{QK^\top}{\sqrt{d_k}} + M \right) V. \quad (4.1)$$

Here  $Q = HW_Q$ ,  $K = HW_K$ , and  $V = HW_V$  are the query, key, and value matrices.

#### Layer 3: derivation

We already introduced attention as a learned weighted average. The matrix form in Eq. (4.1) is just the batched version of Eqs. (2.10)–(2.12). If row  $i$  of  $QK^\top/\sqrt{d_k}$  is the vector of scores  $s_{ij}$  for all  $j$ , then row-wise softmax gives the attention weights for token  $i$ . Multiplying by  $V$  then produces the weighted sum of value vectors.

The computational cost is the important historical detail. Forming  $QK^\top$  costs  $O(L^2d_k)$ , so dense self-attention has quadratic dependence on sequence length  $L$ . This is the starting point from which most later long-context work tries to escape.

#### Algorithmic template

##### Transformer attention, stripped to essentials.

1. Embed tokens into  $H \in \mathbb{R}^{L \times d}$ .
2. Project to  $Q, K, V$ .
3. Compute all pairwise scores  $QK^\top/\sqrt{d_k}$ .
4. Apply the causal mask and row-wise softmax.
5. Mix value vectors using the resulting weights.
6. Repeat across heads and layers.

#### What to remember

The Transformer introduced the basic computational object that almost every later method modifies: *pairwise attention scores over positions*. Long-context work usually changes either the set of positions compared, the way positions are encoded, or the memory from which keys and values are drawn.

## 4.3 Family 2: extending memory beyond one segment

### 4.3.1 Transformer-XL (Dai et al., 2019)

#### Layer 1: intuition

Dense attention only helps inside the current segment. If a document is longer than the segment length, the model still forgets what happened in earlier segments. Transformer-XL adds a simple but powerful idea: reuse hidden states from previous segments as a memory for the next segment.

Suppose the sequence is processed in segments of length  $L$ . Let  $H_\tau^{(\ell-1)} \in \mathbb{R}^{L \times d}$  be the hidden states of segment  $\tau$  at layer  $\ell - 1$ . Transformer-XL stores a memory from the previous segment,

$$\mathcal{M}_{\tau-1}^{(\ell-1)} = \text{SG}(H_{\tau-1}^{(\ell-1)})[-m:], \quad (4.2)$$

where  $m$  is the memory length and SG denotes stop-gradient.

#### Layer 2: core mechanism

For the current segment, queries come from the current hidden states, but keys and values come from the concatenation of old memory and current states:

$$\tilde{H}_\tau^{(\ell-1)} = [\mathcal{M}_{\tau-1}^{(\ell-1)}; H_\tau^{(\ell-1)}]. \quad (4.3)$$

Then

$$Q = H_\tau^{(\ell-1)} W_Q, \quad K = \tilde{H}_\tau^{(\ell-1)} W_K, \quad V = \tilde{H}_\tau^{(\ell-1)} W_V. \quad (4.4)$$

So each token in the current segment can attend not only to the current segment but also to cached states from the previous one.

#### Layer 3: derivation

The more subtle historical contribution of Transformer-XL is its relative-position score decomposition. We want the attention score between current position  $i$  and memory/current position  $j$  to depend not only on content, but also on relative offset  $i - j$ . Transformer-XL writes the logit as

$$A_{ij} = q_i^\top k_j + q_i^\top W_{k,R} r_{i-j} + u^\top k_j + v^\top W_{k,R} r_{i-j}. \quad (4.5)$$

Each term has a meaning.

1.  $q_i^\top k_j$ : content-based addressing.
2.  $q_i^\top W_{k,R} r_{i-j}$ : content-dependent relative-position effect.
3.  $u^\top k_j$ : a global content bias.
4.  $v^\top W_{k,R} r_{i-j}$ : a global positional bias.

This decomposition is historically important because it separates *what the token means* from *where the token lies relative to the query*. Later long-context methods repeatedly revisit this separation.

**Algorithmic template****Transformer-XL segment recurrence.**

1. Process segment  $\tau - 1$  and cache its last  $m$  hidden states in each layer.
2. For segment  $\tau$ , concatenate the cache with the current hidden states when forming keys and values.
3. Use relative-position attention scores as in Eq. (4.5).
4. After processing segment  $\tau$ , store its last  $m$  hidden states for segment  $\tau + 1$ .

**What to remember**

Transformer-XL introduced the important idea that a model can carry state across segments without reprocessing the entire past from scratch.

*Running example.* If page 181 and page 182 of the policy manual fall into different segments, Transformer-XL still lets the later page attend to cached states from the earlier page.

**4.3.2 Compressive Transformer (Rae et al., 2020)****Layer 1: intuition**

Transformer-XL still has a finite memory. Once the cache is full, older states disappear. Compressive Transformer asks: what if old memories are too valuable to throw away, but we can store a compressed version of them?

Compressive Transformer maintains two memories:

- a *short-term memory*  $\mathcal{M}_t$  of recent states, and
- a *compressed memory*  $C_t$  for older states.

When old short-term memory would be evicted, a compression function  $f_c$  converts it into a smaller representation and appends it to  $C_t$ .

**Layer 2: core mechanism**

Let  $\overline{M}_t$  denote the block of short-term memories that is about to be discarded. Then the update is conceptually

$$C_t = [f_c(\overline{M}_t); C_{t-1}^{\text{trim}}], \quad \mathcal{M}_t = [H_t; \mathcal{M}_{t-1}^{\text{trim}}]. \quad (4.6)$$

Attention can then read from the union of current states, short-term memory, and compressed memory.

**Layer 3: derivation**

The key question is how to train the compression function  $f_c$ . Backpropagating through arbitrarily old memories is too expensive. Compressive Transformer therefore introduces a local auxiliary objective that teaches the compressed memory to preserve information useful for later

attention.

One useful abstract form of the auxiliary loss is an attention-reconstruction loss:

$$\mathcal{L}_{\text{comp}} = \sum_h \|\text{Attn}(Q_h, K_h(M), V_h(M)) - \text{Attn}(Q_h, K_h(C), V_h(C))\|_2^2, \quad (4.7)$$

where  $M$  denotes original memory states and  $C = f_c(M)$  denotes their compressed counterpart. The total objective is

$$\mathcal{L} = \mathcal{L}_{\text{LM}} + \lambda \mathcal{L}_{\text{comp}}. \quad (4.8)$$

So the compression network is not asked to preserve *all* information uniformly. It is asked to preserve information that matters to downstream attention.

### What to remember

Compressive Transformer introduced a principle that reappears in many modern memory systems: *very old context can be summarized instead of kept verbatim*. Modern KV-cache compression and memory consolidation ideas are descendants of this principle.

## 4.4 Family 3: reducing the quadratic cost of attention

### 4.4.1 Reformer (Kitaev et al., 2020)

#### Layer 1: intuition

Dense attention compares every token with every other token, which is expensive for long sequences. Reformer speeds this up by approximating the search for relevant keys: similar queries and keys are hashed into the same buckets so that attention is computed mostly within buckets instead of across the full sequence.

#### Layer 2: core mechanism

The core trick is locality-sensitive hashing (LSH). A random projection matrix  $R$  maps a vector  $x$  to hash buckets. One simple LSH-style form is

$$h(x) = \arg \max([xR, -xR]), \quad (4.9)$$

so vectors with similar directions tend to land in the same bucket. Reformer sorts tokens by bucket ID and performs attention within buckets (plus nearby chunks for safety) rather than across the full sequence.

#### Layer 3: derivation

Why does this help? Suppose each bucket contains about  $B$  tokens. Then attention cost becomes roughly  $O(LB)$  instead of  $O(L^2)$ , plus sorting overhead  $O(L \log L)$ . This is beneficial when  $B \ll L$ .

The approximation logic is simple. In dense attention, a query mostly needs access to high-

similarity keys. LSH is a fast approximate-nearest-neighbor heuristic. If the buckets place similar vectors together with high probability, then most of the important attention mass can be recovered from within-bucket attention.

#### Common confusion

Reformer is an efficiency paper first, not a position-extrapolation paper. It reduces attention cost; it does not by itself solve the out-of-distribution positional problem that later RoPE-scaling work focuses on.

### 4.4.2 Longformer (Beltagy et al., 2020)

#### Layer 1: intuition

Longformer was designed for long documents such as articles and reports. Its main idea is that most tokens do not need to look everywhere. They mostly need local context, while a small set of special tokens should have global reach.

#### Layer 2: core mechanism

Let  $w$  be the local window size and  $G$  the set of global tokens. The attention mask is

$$M_{ij} = \begin{cases} 0, & |i - j| \leq w \text{ or } i \in G \text{ or } j \in G, \\ -\infty, & \text{otherwise.} \end{cases} \quad (4.10)$$

Thus ordinary tokens attend locally, while global tokens attend broadly.

#### Layer 3: derivation

With the mask in Eq. (4.10), each non-global token attends to about  $2w + 1$  positions rather than  $L$  positions, so the main cost becomes  $O(Lw)$  rather than  $O(L^2)$ . If the number of global tokens is  $|G|$ , the total cost is roughly  $O(Lw + L|G|)$ .

This sparse pattern is hand-designed, not learned from scratch. That design choice is important historically: Longformer showed that domain structure (documents are often locally coherent) can be built directly into the attention pattern.

### 4.4.3 BigBird (Zaheer et al., 2020)

#### Layer 1: intuition

BigBird generalizes the sparse-attention idea by combining three edge types in the attention graph: local edges, global edges, and random edges. The random edges help information propagate across the sequence more flexibly.

**Layer 2: core mechanism**

If token  $i$  attends to a local neighborhood, to a small global set, and to  $r$  random positions, then the sparse mask gives each token about  $O(w + |G| + r)$  connections. The total cost becomes roughly

$$O(L(w + |G| + r)). \quad (4.11)$$

**Layer 3: derivation**

The key conceptual derivation is graph-based. Dense attention corresponds to a complete graph on positions. BigBird approximates this with a sparse graph that still has short path lengths and global communication channels. The random edges reduce the risk that information gets trapped in isolated local neighborhoods. This is why BigBird could provide both strong empirical performance and more comforting theoretical arguments than purely local sparsity.

**What to remember**

Reformer, Longformer, and BigBird are historically important because they attacked the *quadratic cost bottleneck*. They are not the main ancestors of RoPE-scaling methods, but they established the long-sequence mindset that not every token pair must interact directly.

*Running example.* In the policy-manual task, a Longformer-style model would mostly compare each sentence with nearby sentences, while a few global tokens such as section headers or query tokens would remain broadly visible.

## 4.5 Family 4: position handling and extrapolation

### 4.5.1 ALiBi (Press et al., 2022)

**Layer 1: intuition**

ALiBi uses a strikingly simple idea: instead of learning or rotating position vectors, directly penalize attention to distant tokens by subtracting a distance-dependent bias from the attention score. The farther away a token is, the more its score is reduced.

**Layer 2: core mechanism**

For head  $h$  with slope  $m_h > 0$ , ALiBi changes the attention score to

$$s_{ij}^{(h)} = \frac{q_i^\top k_j}{\sqrt{d_k}} - m_h(i - j), \quad j \leq i. \quad (4.12)$$

Different heads use different slopes, so some heads strongly prefer nearby tokens while others decay more slowly with distance.

**Layer 3: derivation**

Why does ALiBi extrapolate better than learned absolute position embeddings? Because Eq. (4.12) is defined for any distance  $i - j$ , not just distances seen during training. The model

is biased toward locality, but the rule itself does not break when the sequence becomes longer. It continues to assign a valid bias to every new distance.

#### 4.5.2 RoPE / RoFormer (Su et al., 2021)

##### Layer 1: intuition

RoPE injects position not by adding a position vector, but by rotating pairs of coordinates in the query and key vectors. The elegant part is that the resulting inner product depends on *relative* position difference automatically.

##### Layer 2: core mechanism

For one 2D coordinate pair and frequency  $\theta$ , define the rotation matrix

$$R(m\theta) = \begin{bmatrix} \cos(m\theta) & -\sin(m\theta) \\ \sin(m\theta) & \cos(m\theta) \end{bmatrix}. \quad (4.13)$$

If  $q$  and  $k$  are the unrotated query/key coordinate pairs at positions  $m$  and  $n$ , RoPE uses rotated versions

$$\tilde{q}_m = R(m\theta)q, \quad \tilde{k}_n = R(n\theta)k. \quad (4.14)$$

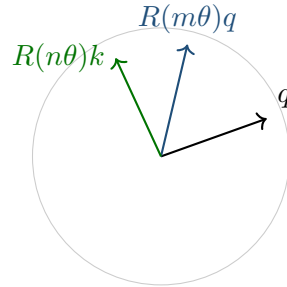
##### Layer 3: derivation

The key identity is

$$\tilde{q}_m^\top \tilde{k}_n = q^\top R(m\theta)^\top R(n\theta)k \quad (4.15)$$

$$= q^\top R((n-m)\theta)k. \quad (4.16)$$

Because  $R(a)^\top R(b) = R(b-a)$ , the score depends only on the relative position difference  $n-m$ . For the full vector, RoPE applies this 2D rotation independently to many coordinate pairs using different frequencies  $\theta_1, \dots, \theta_{d/2}$ . Therefore the total attention logit encodes a multi-frequency representation of relative displacement.



RoPE rotates coordinate pairs by angles proportional to position.

**Figure 3:** A cartoon of RoPE in one 2D subspace. Relative phase, not absolute coordinates alone, drives the final query–key interaction.

**What to remember**

ALiBi and RoPE are the two most important historical position-handling branches for later long-context extension. ALiBi adds distance penalties directly; RoPE rotates coordinates so relative displacement appears inside the inner product.

**4.6 Family 5: external evidence and retrieval as memory****4.6.1 REALM (Gua et al., 2020)****Layer 1: intuition**

REALM treats retrieved documents as latent variables during pretraining. The model does not assume it already knows everything in its parameters; it learns to fetch potentially relevant documents from an external index.

**Layer 2: core mechanism**

Let  $x$  be the input and  $z$  a retrieved document. REALM writes the prediction as a latent-document marginal:

$$p(y | x) = \sum_{z \in \mathcal{D}} p_{\eta}(z | x) p_{\theta}(y | x, z). \quad (4.17)$$

Here  $p_{\eta}(z | x)$  is the retriever distribution and  $p_{\theta}(y | x, z)$  is the reader or generator.

**Layer 3: derivation**

Eq. (4.17) is just the law of total probability with a hidden document variable  $z$ . The difficulty is computational: summing over all documents is impossible for large corpora. REALM therefore approximates the marginal with top- $k$  retrieval:

$$p(y | x) \approx \sum_{z \in \text{top-}k(p_{\eta}(\cdot | x))} p_{\eta}(z | x) p_{\theta}(y | x, z). \quad (4.18)$$

This latent-variable view is a conceptual ancestor of much later RAG-style work.

**4.6.2 RAG (Lewis et al., 2020)****Layer 1: intuition**

RAG brought the latent-document view directly into generation. Instead of hoping the model remembers everything in its parameters, it allows the answer to depend explicitly on retrieved documents.

**Layer 2: core mechanism**

A common sequence-level RAG formulation is

$$p(y \mid x) = \sum_{z \in \mathcal{D}} p_{\eta}(z \mid x) p_{\theta}(y \mid x, z). \quad (4.19)$$

A token-level variant allows document weights to change during generation:

$$p(y_t \mid x, y_{<t}) = \sum_{z \in \mathcal{D}} p_{\eta}(z \mid x) p_{\theta}(y_t \mid x, z, y_{<t}). \quad (4.20)$$

**Layer 3: derivation**

RAG inherits the same top- $k$  approximation logic as REALM. If the retriever returns  $z_{1:k}$ , then generation can be approximated by

$$\log p(y \mid x) \approx \log \sum_{i=1}^k p_{\eta}(z_i \mid x) p_{\theta}(y \mid x, z_i). \quad (4.21)$$

This formulation clarifies a deep principle: retrieval augmentation is not just prompt engineering. It changes the conditional distribution being modeled by making evidence an explicit variable.

**4.6.3 FiD (Izacard and Grave, 2021)****Layer 1: intuition**

FiD stands for *Fusion-in-Decoder*. Instead of concatenating all retrieved passages into one giant encoder input, it encodes each passage separately and lets the decoder fuse them through cross-attention.

**Layer 2: core mechanism**

If  $z_1, \dots, z_k$  are retrieved passages, FiD encodes each passage independently to obtain encoder states  $E_1, \dots, E_k$ . The decoder then cross-attends to the concatenated encoder memories:

$$\text{Attn}_{\text{cross}}(Q, [K(E_1); \dots; K(E_k)], [V(E_1); \dots; V(E_k)]). \quad (4.22)$$

**Layer 3: derivation**

FiD does not marginalize document probabilities the way RAG does. Its insight is architectural: keep passage encodings separate until the decoder decides how to combine them. This delays fusion and often works better when several passages each contain part of the answer.

#### 4.6.4 RETRO (Borgeaud et al., 2022)

##### Layer 1: intuition

RETRO moves retrieval even closer to the language-model loop. Instead of retrieving once for the whole query, it retrieves neighbors for chunks of the current context and conditions generation on those neighbors.

##### Layer 2: core mechanism

Let the current input be broken into chunks  $c_1, c_2, \dots$ . For each chunk  $c_t$ , retrieve nearest-neighbor chunks  $n_{t,1}, \dots, n_{t,k}$ . The model predicts the next token conditioned on both the local context and retrieved neighbors:

$$p(y_t \mid y_{<t}, n_{1:k}(c_{\leq t})). \quad (4.23)$$

Cross-attention layers inject the retrieved neighbors into the model at multiple depths.

##### Layer 3: derivation

The chunk-based formulation matters because it turns retrieval into a repeated local operation. That makes RETRO closer to a learned memory hierarchy than to classic one-shot retrieval.

#### 4.6.5 Memorizing Transformers (Wu et al., 2022)

##### Layer 1: intuition

Memorizing Transformers add a nearest-neighbor external memory directly to attention. The model can look not only at the current context window but also at a much larger store of past key–value pairs.

##### Layer 2: core mechanism

Let the current query set be  $Q$  and the external memory contain key–value pairs  $(k_r^{\text{mem}}, v_r^{\text{mem}})$ . For each query  $q_i$ , retrieve the top- $k$  nearest memory keys and augment the candidate attention set:

$$\mathcal{N}(q_i) = \text{top-}k_r \ q_i^\top k_r^{\text{mem}}. \quad (4.24)$$

Attention is then computed over both in-window keys and retrieved memory keys.

##### What to remember

REALM, RAG, FiD, RETRO, and Memorizing Transformers established retrieval and external memory as first-class ingredients of language modeling. Modern grounded-generation work builds directly on this branch.

*Running example.* Instead of forcing the model to read the full 300-page manual every time, this family retrieves the incident-reporting rule, the sign-off clause, and the exception section, then conditions the answer on those pieces.

## 4.7 Historical synthesis: what the roots gave us

| Family            | Core question                                                                           | Descendants in modern work                                                                       |
|-------------------|-----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| Dense attention   | How should tokens share information within one context window?                          | All later transformer variants.                                                                  |
| Memory extension  | How can the model keep older information without rereading everything?                  | KV caching, streaming models, memory compression, finite-scope infinite-context methods.         |
| Cost reduction    | How can we avoid $O(L^2)$ attention cost?                                               | Sparse attention, block attention, approximate attention, chunking.                              |
| Position geometry | How can attention scores remain meaningful at unseen positions?                         | PI, YaRN, PoSE, LongRoPE, LongRoPE2, many long-context adaptation recipes.                       |
| External evidence | How can the model use retrieved evidence rather than relying only on parametric memory? | Self-RAG, CRAG, GraphRAG, LongCite, citation alignment, hybrid long-context + retrieval systems. |

**Table 4:** The modern field is easier to understand once the roots are grouped by the bottleneck they tried to solve.

## 5 Common Framework A: position-geometry preservation for long-context stability

### Plain-language summary

This framework explains a large fraction of modern long-context extension papers. The starting insight is simple: if a model was trained mostly on positions up to, say, 4k or 8k tokens, then feeding it positions far beyond that range creates an out-of-distribution positional regime. The model may accept the longer input, but the geometry of attention scores can become unstable or unfamiliar.

### 5.1 Why RoPE-based models often break when context becomes very long

#### Layer 1: intuition

RoPE rotates each query and key coordinate pair by an angle proportional to token position. If test-time positions become much larger than the training positions, these angles also become much larger. The model then sees relative phases it was not trained to use reliably.

Let  $m$  and  $n$  be token positions. In one frequency subspace, RoPE gives the interaction

$$\tilde{q}_m^\top \tilde{k}_n = q^\top R((n - m)\theta)k. \quad (5.1)$$

So the score depends on the relative phase  $(n - m)\theta$ . If training only exposed the model to

$|n - m| \leq L_{\text{train}}$ , then very large test-time offsets create phase patterns outside that familiar range.

#### Layer 2: core mechanism

A useful abstract view is this. Let  $\phi(m)$  denote the position-dependent transformation applied inside attention. Long-context failure appears when

$$\phi(m), \phi(n), \text{ or } \phi(n - m) \quad (5.2)$$

for test positions  $m, n$  fall outside the region where training made attention scores well-behaved. For RoPE, that region is governed by the frequencies and the observed position range during training.

#### Common confusion

**The problem is not only memory size.** Even if hardware can store all tokens, the model can still fail because the attention-score geometry at very large positions is poorly calibrated.

## 5.2 A generic remapping view

Many long-context methods can be written as position remapping. Instead of feeding the raw position  $m$  into the positional mechanism, use a transformed position  $g(m)$ .

#### Layer 2: core mechanism

If the original attention score uses positional transformation  $\phi(m)$ , the remapped score uses  $\phi(g(m))$ . The design problem is therefore

$$\text{choose } g \text{ so that } g(m) \approx m \text{ on short contexts but compresses or regularizes large positions.} \quad (5.3)$$

This single equation unifies PI, YaRN, PoSE, LongRoPE, and several related methods.

#### Layer 3: derivation

A good remapping function should satisfy three competing goals.

1. **Short-context fidelity:** if  $m$  lies in the original training range, then  $g(m)$  should not distort it too much.
2. **Long-context compression:** if  $m$  is far beyond the training range, then  $g(m)$  should keep the induced attention phases in a safer region.
3. **Optimization compatibility:** the remapped model should adapt with limited continued training rather than requiring full pretraining from scratch.

So long-context extension becomes a constrained geometry-preservation problem.

## 5.3 Positional Interpolation (PI): the canonical first modern recipe

### 5.3.1 Layer 1: intuition

#### Layer 1: intuition

If a model was trained up to  $L_{\text{train}} = 4\text{k}$  tokens but we want  $L_{\text{test}} = 32\text{k}$  tokens, one naive strategy is to keep using raw positions  $0, 1, 2, \dots, 31999$ . PI says: do not do that. Instead, *compress* the test-time positions back into the familiar training range before applying RoPE.

### 5.3.2 Layer 2: core mechanism

#### Layer 2: core mechanism

The simplest PI map is linear:

$$g_{\text{PI}}(m) = m \cdot \frac{L_{\text{train}}}{L_{\text{test}}}. \quad (5.4)$$

Then RoPE is computed using  $g_{\text{PI}}(m)$  instead of  $m$ .

### 5.3.3 Layer 3: derivation

#### Layer 3: derivation

Why does Eq. (5.4) help? In one frequency subspace, the RoPE relative phase becomes

$$(g_{\text{PI}}(n) - g_{\text{PI}}(m))\theta = (n - m)\theta \frac{L_{\text{train}}}{L_{\text{test}}}. \quad (5.5)$$

So all relative phases are shrunk by a factor  $L_{\text{train}}/L_{\text{test}}$ . Large unseen offsets in the raw position space are mapped to more familiar offsets in the rotated coordinate space.

This immediately reduces score volatility. If raw extrapolation can create very large phase differences and therefore unstable score patterns, interpolation dampens those differences. That is why PI often works with modest continued training.

#### Algorithmic template

##### PI in five steps.

1. Choose the desired target context length  $L_{\text{test}}$ .
2. For each token position  $m$ , compute the compressed index  $g_{\text{PI}}(m)$ .
3. Apply RoPE using  $g_{\text{PI}}(m)$  rather than  $m$ .
4. Continue training or fine-tune on long examples.
5. Evaluate not only at long length, but also on short-context tasks to check regression.

#### What to remember

PI is historically important because it gave the field a simple, stable, and reusable starting point for RoPE-based long-context extension [19].

## 5.4 YaRN: interpolation plus targeted correction

### Layer 1: intuition

PI compresses positions uniformly. YaRN observes that uniform compression is helpful but not always enough. Different frequency bands and distance regimes may need different treatment, and continued training should be used more efficiently.

### Layer 2: core mechanism

A useful abstract description of YaRN is that it uses a *piecewise or frequency-aware interpolation schedule* together with scaled training so that the model adapts to the new position regime using fewer long-context tokens [22]. In simplified form, one can write

$$g_{\text{YaRN}}(m; \theta_j) = \alpha_j(m) m, \quad (5.6)$$

where the shrinkage factor  $\alpha_j(m)$  may depend on both the frequency band  $\theta_j$  and the position range.

### Layer 3: derivation

Why is frequency awareness natural? In RoPE, different coordinate pairs use different frequencies. High-frequency pairs rotate faster, so they are more sensitive to large position indices. A single global compression ratio may therefore over-correct some bands and under-correct others. Allowing the remapping to vary by band or range lets the method preserve more short-context structure while taming the most unstable long-range phases.

The resulting objective is still the same as in Eq. (5.3): preserve useful short-range geometry and regularize long-range geometry. YaRN can be viewed as a more refined solution to the same design problem.

## 5.5 PoSE, LongLoRA, and LongAlign: make adaptation cheaper

### Plain-language summary

PI and YaRN say *what positional change to make*. PoSE, LongLoRA, and LongAlign say *how to adapt the model efficiently once we know the target long-context regime*. They are therefore best seen as training recipes that sit inside Framework A rather than as unrelated ideas.

### 5.5.1 PoSE

#### Layer 1: intuition

PoSE tries to teach the model long-range positional behavior without always training on full-length long sequences. It simulates long positional gaps inside shorter training windows by skipping position indices between chunks.

**Layer 2: core mechanism**

If a training example is divided into chunks, PoSE assigns chunk-specific position offsets so that the model observes effective positions spread across a much larger target range even though the actual training sequence is short [23].

**Layer 3: derivation**

In our remapping notation, PoSE does not mainly change the inference-time map  $g(m)$ . Instead, it changes the *training distribution over positions*. It samples examples so the model learns to operate on a wider set of effective positional indices. This directly reduces the positional distribution shift between training and test.

**5.5.2 LongLoRA****Layer 1: intuition**

LongLoRA is a practical recipe for extending open-weight LLMs with limited hardware. It uses sparse local attention during training and parameter-efficient adaptation so that the cost of long-context fine-tuning does not explode.

**Layer 2: core mechanism**

Its main training trick is shifted sparse attention during adaptation, while keeping dense attention at inference if desired. Abstractly, LongLoRA optimizes

$$\min_{\Delta\theta} \mathcal{L}_{\text{LM}}(\theta_0 + \Delta\theta; g(m), \text{long data}), \quad (5.7)$$

where  $\Delta\theta$  is low-rank or otherwise parameter-efficient, and  $g(m)$  is a long-context positional strategy such as interpolation [24].

**5.5.3 LongAlign****Layer 1: intuition**

LongAlign emphasizes that long-context ability is not only a positional problem. If instruction tuning never uses long, instruction-following examples, the model may still behave poorly in long contexts even after positional extension. LongAlign therefore focuses on long instruction data, packing, batching, and loss balancing [30].

**What to remember**

PoSE broadens the positional training distribution. LongLoRA reduces adaptation cost. LongAlign improves long-context instruction alignment. All three are best understood as training-time support mechanisms for the same underlying position-geometry problem.

## 5.6 LongRoPE and LongRoPE2: nonuniform search and progressive extension

### 5.6.1 LongRoPE

#### Layer 1: intuition

LongRoPE asks a bold question: can we push RoPE-based models to million-scale windows without retraining from scratch? Its answer is to use nonuniform interpolation schedules, search for good scaling factors, and extend progressively rather than in one jump.

#### Layer 2: core mechanism

In abstract form, LongRoPE uses a searched remapping

$$g_{\text{LR}}(m; \psi), \quad (5.8)$$

where  $\psi$  contains per-dimension or per-range scaling factors. The method first extends to an intermediate long context (for example 128k or 256k), fine-tunes briefly, and then extends further using the already-adapted checkpoint [31].

#### Layer 3: derivation

The logic behind progressive extension is straightforward. A direct jump from 4k to 2M forces the model to absorb a huge positional shift in one step. A staged procedure reduces the gap at each stage:

$$4\text{k} \rightarrow 128\text{k} \rightarrow 256\text{k} \rightarrow 2\text{M}. \quad (5.9)$$

At each stage, the remapping parameters can be re-optimized, and the model weights only need to adapt to a smaller incremental shift. LongRoPE therefore combines *search in positional space* with *curriculum in training space*.

### 5.6.2 LongRoPE2

#### Layer 1: intuition

LongRoPE2 sharpens the diagnosis. It argues that some higher RoPE dimensions were under-trained for long-distance behavior, so merely rescaling positions is not enough. It therefore combines improved rescaling with mixed-window training that preserves short-context behavior [40].

#### Layer 2: core mechanism

A useful abstract objective is

$$\min_{\psi, \theta} \mathcal{L}_{\text{long}}(\theta; g_{\psi}(m)) + \beta \mathcal{L}_{\text{short}}(\theta; m), \quad (5.10)$$

where the first term encourages good long-context behavior under a searched remapping  $g_\psi$  and the second term preserves the original short-context quality.

### Layer 3: derivation

Eq. (5.10) makes explicit a trade-off that many long-context methods quietly face. If we modify positional behavior too aggressively, we may damage short-context performance. LongRoPE2 turns that trade-off into a two-term optimization problem and uses mixed context windows during training to keep both regimes alive.

## 5.7 Complementary ideas: StreamingLLM, Infini-attention, and ReAttention

### Plain-language summary

Not every recent long-context paper is primarily about RoPE scaling. Some papers instead change how attention or memory is executed at inference time. These methods are complementary to Framework A because they address a second bottleneck: finite attention scope and memory budget.

**StreamingLLM** (2023/2024): keep a small set of “attention sink” tokens plus a rolling local cache during streaming inference. *Why it mattered:* it showed that surprisingly small anchored memories can stabilize long-form streaming generation [25].

**Infini-attention** (2024): augment local attention with a compressive memory mechanism that can in principle grow without a fixed global window. *Why it mattered:* it revived the memory-extension branch in the LLM era [29].

**ReAttention** (2024/2025): perform a position-agnostic top- $k$  selection before ordinary position-aware attention. *Why it mattered:* it offered a training-free path to much longer effective contexts by reducing dependence on full-range positional extrapolation [39].

### What to remember

Framework A is widest when interpreted correctly. It includes not only *how positions are remapped*, but also *how long-range memory is executed without blowing up attention cost or positional drift*.

## 5.8 A unified derivation of the final algorithmic template for Framework A

### Layer 3: derivation

We can now derive a generic long-context extension algorithm.

**Step 1: start from a base RoPE model.** The base attention logit for head  $h$  is

$$\ell_{ij}^{(h)} = \frac{\langle \text{RoPE}(q_i, m_i), \text{RoPE}(k_j, m_j) \rangle}{\sqrt{d_k}} + b_{ij}^{(h)}, \quad (5.11)$$

where  $b_{ij}^{(h)}$  may include masks or extra biases.

**Step 2: introduce a remapping function.** Replace raw positions by transformed positions  $g_\psi(m)$ :

$$\ell_{ij}^{(h)}(\psi) = \frac{\langle \text{RoPE}(q_i, g_\psi(m_i)), \text{RoPE}(k_j, g_\psi(m_j)) \rangle}{\sqrt{d_k}} + b_{ij}^{(h)}. \quad (5.12)$$

**Step 3: choose how  $g_\psi$  is parameterized.**

- PI: one global linear compression factor.
- YaRN: range-aware or frequency-aware interpolation schedule.
- LongRoPE: searched nonuniform schedule, often staged.
- LongRoPE2: searched schedule plus mixed short/long training.

**Step 4: choose the training distribution.** This can include ordinary long examples, skip-wise position sampling as in PoSE, parameter-efficient adaptation as in LongLoRA, and long instruction data as in LongAlign.

**Step 5: optionally modify execution.** Add memory compression, finite-scope attention, streaming anchors, or training-free preselection if the target context is extremely large.

**Step 6: optimize a two-regime objective.** A useful generic objective is

$$\min_{\psi, \theta} \underbrace{\mathcal{L}_{\text{long}}(\theta, \psi)}_{\text{extend the context}} + \beta \underbrace{\mathcal{L}_{\text{short}}(\theta, \psi)}_{\text{preserve existing quality}}. \quad (5.13)$$

The exact form differs by paper, but the logic is the same: extend length without breaking the base model.

### Algorithmic template

#### Framework A: final recipe.

1. Diagnose the base model’s trained context length and positional mechanism.
2. Choose a position remapping  $g_\psi$  that preserves short-range geometry and compresses long-range geometry.
3. Select a cost-feasible adaptation strategy: full continued pretraining, PoSE-like synthetic long positions, LongLoRA-style parameter-efficient fine-tuning, or staged extension.
4. Include a short-context preservation term or mixed-window training.
5. If the target length is extreme, add memory or finite-scope execution tricks.
6. Evaluate on both short- and long-context benchmarks, and test sensitivity to answer position, not only total length.

## 5.9 How the community learned to evaluate Framework A

**Lost in the Middle** (2023/2024): showed that many models use long contexts unevenly and struggle when evidence is in the middle. *Why it mattered*: it changed evaluation culture from “can

the model fit the tokens?” to “where in the context does the model actually succeed?” [20]..

**LongBench** (2023/2024): offered the first broad multilingual benchmark for long-context understanding. *Why it mattered:* it gave researchers a shared multi-task testbed [21]..

**RULER** (2024): stress-tested effective context size beyond simple needle retrieval. *Why it mattered:* it showed that nominal window size often overstates real usable length [28]..

**$\infty$ Bench** (2024): pushed benchmark lengths beyond 100k tokens. *Why it mattered:* it exposed the gap between synthetic passkey success and realistic long-context performance [27]..

**LongBench v2** (2024/2025): made long-context reasoning more realistic and difficult. *Why it mattered:* it shifted attention toward deeper reasoning rather than shallow retrieval alone [38]..

### What to remember

Framework A is not just a bag of RoPE hacks. It is a coherent design problem: keep attention geometry meaningful as positions grow, while preserving short-context ability and controlling computation.

## 6 Common Framework B: evidence-grounded selective generation

### Plain-language summary

This framework explains a large fraction of hallucination-reduction methods. The core idea is that the model should not freely generate from a vague mixture of parametric memory and noisy context. It should instead *select, compress, verify, and cite* evidence in a controlled way.

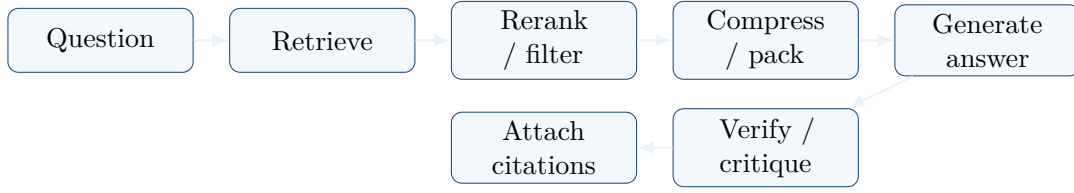
### 6.1 The end-to-end problem in plain language

#### Layer 1: intuition

Suppose the corpus contains ten million tokens and the answer depends on two short paragraphs. A faithful system must answer four questions in order.

1. Do I need external evidence at all?
2. If yes, which passages are worth retrieving?
3. If I retrieved many passages, which ones should actually influence the answer?
4. Once I answer, can I verify and cite the support for each claim?

This is why grounded generation is a pipeline problem, not just a decoding problem.



**Figure 4:** Evidence-grounded selective generation pipeline. Different papers improve different stages.

## 6.2 Latent-evidence formulation

### Layer 2: core mechanism

Let  $x$  be the query and let  $z$  denote the evidence unit selected by the system. Then the generic latent-variable formulation is

$$p(y | x) = \sum_{z \in \mathcal{Z}} p_{\eta}(z | x) p_{\theta}(y | x, z), \quad (6.1)$$

where  $\eta$  are retriever parameters and  $\theta$  are reader/generator parameters.

### Layer 3: derivation

Eq. (6.1) is the same law-of-total-probability logic we saw in REALM and RAG, but now we reinterpret it as a framework for many later methods.

**Why is  $z$  called a latent variable?** Because the model does not know in advance which evidence unit will ultimately be most useful. That hidden choice is represented by  $z$ .

**Why is the full sum intractable?** Because the evidence space  $\mathcal{Z}$  may include millions of passages or graphs. Therefore most practical systems approximate Eq. (6.1) using top- $k$  selection:

$$p(y | x) \approx \sum_{z \in \text{top-}k(p_{\eta}(\cdot | x))} p_{\eta}(z | x) p_{\theta}(y | x, z). \quad (6.2)$$

**Where does hallucination enter?** Hallucination arises when the selected evidence is wrong or incomplete, or when  $p_{\theta}(y | x, z)$  generates claims not supported by  $z$ .

## 6.3 Retrieval, reranking, and compression as optimization subproblems

### 6.3.1 Retrieval

#### Layer 2: core mechanism

The retriever solves

$$z_{1:k} = \text{top-}k_{z \in \mathcal{Z}} r_{\eta}(x, z), \quad (6.3)$$

where  $r_{\eta}$  is a relevance score. The retriever’s job is *recall*: do not miss the useful evidence.

### 6.3.2 Reranking

#### Layer 2: core mechanism

Given an initial candidate set  $z_{1:k}$ , a reranker computes a stronger score

$$\rho_\phi(x, z_i) \tag{6.4}$$

and reorders the candidates. The reranker’s job is *precision*: push the best evidence to the top.

### 6.3.3 Compression / packing

#### Layer 2: core mechanism

If the selected evidence exceeds the prompt budget  $\mathcal{B}$ , the system must choose a compressed representation  $c(z_{1:k})$  satisfying

$$|c(z_{1:k})| \leq \mathcal{B}, \tag{6.5}$$

while keeping answer-relevant information. This can mean extractive span selection, abstractive summarization, or passage truncation with learned ordering.

#### Common confusion

**Compression is not automatically safe.** A compressed summary may discard a small but crucial exception clause. This is why retrieval pipelines often need separate handling for “main rule” versus “exception” evidence.

## 6.4 From evidence selection to faithful answers

#### Layer 1: intuition

Even with perfect retrieval, the model can still summarize the evidence incorrectly. So grounded generation needs a second layer of control: the answer should be *conditioned by evidence* and ideally *checked against evidence*.

#### Layer 2: core mechanism

A useful faithfulness-regularized objective is

$$\mathcal{L}(\theta, \eta) = \mathcal{L}_{\text{ans}} + \lambda \mathcal{L}_{\text{faith}} + \gamma \mathcal{L}_{\text{cite}}, \tag{6.6}$$

where

- $\mathcal{L}_{\text{ans}}$  rewards task correctness,
- $\mathcal{L}_{\text{faith}}$  penalizes unsupported claims,
- $\mathcal{L}_{\text{cite}}$  rewards accurate citation spans.

Different modern papers instantiate these terms differently, but this decomposition unifies a lot

of the literature.

## 6.5 Verification and revision: RARR, CoVe, and SelfCheckGPT

**RARR** (2023): ask an LM to research and revise its own output using external evidence. *Why it mattered*: it made post-hoc factual revision a practical pipeline idea [15]..

**Chain-of-Verification (CoVe)** (2023/2024): decompose verification into explicit checking steps before finalizing the answer. *Why it mattered*: it showed that structured self-checking can reduce hallucination [26]..

**SelfCheckGPT** (2023): detect hallucination by measuring inconsistency across multiple sampled generations. *Why it mattered*: it offered a zero-resource black-box detection method when retrieval labels are unavailable [16]..

### What to remember

These works fit Framework B because they try to reduce unsupported generation after or during answer formation, even if they do not change the retriever itself.

## 6.6 Adaptive retrieval and correction: Self-RAG and CRAG

### 6.6.1 Self-RAG

#### Layer 1: intuition

Classic RAG often retrieves a fixed number of passages whether retrieval is needed or not. Self-RAG makes retrieval and critique *adaptive*. The model can decide when to retrieve and can emit reflection signals about the quality of evidence and answers.

#### Layer 2: core mechanism

A simplified abstract view is that Self-RAG augments generation with control tokens for retrieval and critique. At each stage, the model may choose an action  $a_t \in \{\text{retrieve, continue, critique}\}$  and then condition subsequent generation on both evidence and reflection tokens [32].

#### Layer 3: derivation

In Framework B language, Self-RAG learns a policy over latent evidence actions. Rather than treating retrieval as a fixed pre-step, it embeds retrieval choice and critique into the generation process itself. The implicit objective is not just Eq. (6.2); it is a policy-conditioned version in which the model decides whether the evidence variable  $z$  should be introduced at all, and how strongly the final answer should depend on it.

### 6.6.2 CRAG

#### Layer 1: intuition

CRAG observes that retrieval quality varies. If retrieved evidence is bad, blindly conditioning on it can hurt. So CRAG adds a correction stage: estimate retrieval quality, refine the evidence set, and only then generate.

#### Layer 2: core mechanism

A useful abstract decomposition is

$$\text{query } x \rightarrow \text{retrieve } z_{1:k} \rightarrow \text{evaluate quality } q(z_{1:k} | x) \rightarrow \text{correct } \hat{z}_{1:k} \rightarrow \text{generate } y. \quad (6.7)$$

Thus CRAG explicitly models retrieval reliability before answer generation [33].

#### What to remember

Self-RAG makes retrieval adaptive *inside* generation. CRAG makes retrieval robust *before* generation by checking and correcting evidence quality.

## 6.7 Structured retrieval: GraphRAG, LongRAG, and R<sup>2</sup>AG

### 6.7.1 GraphRAG

#### Layer 1: intuition

Ordinary passage retrieval works well for local fact questions. It is weaker for global corpus questions such as “What are the main themes across this whole collection?” GraphRAG addresses this by building an entity graph and community summaries before question time.

#### Layer 2: core mechanism

Abstractly, GraphRAG builds an intermediate graph representation

$$G = (V, E) \quad (6.8)$$

from the corpus, clusters the graph into communities, summarizes each community, and then answers a query by combining community-level partial answers [34].

#### Layer 3: derivation

Framework B usually selects evidence directly from raw passages. GraphRAG inserts a structure-building map

$$\mathcal{Z}_{\text{raw}} \rightarrow G \rightarrow \mathcal{Z}_{\text{summary}} \quad (6.9)$$

before generation. This is useful when the relevant answer is a corpus-level synthesis rather than a single extractive fact.

### 6.7.2 LongRAG and R<sup>2</sup>AG

**LongRAG** (2024): combine retrieval with long-context models instead of treating them as opposites. *Why it mattered:* it emphasized that long windows and retrieval are complementary tools [35]..

**R<sup>2</sup>AG** (2024): inject richer retrieval-side signals into generation. *Why it mattered:* it highlighted that the generator should not only see text passages, but also retrieval information itself [36]..

## 6.8 Citation as a first-class objective: LongCite and SelfCite

### 6.8.1 LongCite

#### Layer 1: intuition

LongCite turns citation generation into a direct supervised target. Instead of hoping that a good answer will somehow yield a good citation, it trains models to produce answer sentences together with fine-grained supporting spans.

#### Layer 2: core mechanism

Let the answer contain sentences  $s_1, \dots, s_m$ . LongCite learns a model that outputs pairs

$$(s_j, \hat{z}_j), \quad j = 1, \dots, m, \quad (6.10)$$

where  $\hat{z}_j$  is the supporting citation span for sentence  $s_j$  [37].

#### Layer 3: derivation

LongCite fits naturally into Eq. (6.6). The supervision does not only cover answer text; it also covers the span alignment variable. This directly trains the model toward lower citation risk  $R_{\text{cite}}$  from Eq. (3.4).

### 6.8.2 SelfCite

#### Layer 1: intuition

Human citation labels are expensive. SelfCite asks whether a model can teach itself better citation behavior by checking necessity and sufficiency through context ablation.

#### Layer 2: core mechanism

Let  $c$  be a claim and  $z$  a candidate citation span. SelfCite evaluates two ideas:

1. **Necessity:** if  $z$  is removed, the model should struggle to regenerate the same claim.
2. **Sufficiency:** if only  $z$  is retained, the model should still be able to regenerate the claim.

These signals define a self-supervised reward for citation quality [42].

**Layer 3: derivation**

A simple abstract reward is

$$R_{\text{cite}}(c, z) = \underbrace{[p(c \mid z) - p(c \mid \emptyset)]}_{\text{sufficiency}} + \underbrace{[p(c \mid E) - p(c \mid E \setminus z)]}_{\text{necessity}}. \quad (6.11)$$

Best-of- $N$  decoding or preference optimization can then maximize expected citation reward. This is a clean modern example of turning citation into an optimization target rather than a formatting afterthought.

**6.9 A unified derivation of the final algorithmic template for Framework B****Layer 3: derivation**

We now derive a generic faithful-generation system.

**Step 1: retrieve candidates.** Given a query  $x$ , compute initial evidence candidates using Eq. (6.3).

**Step 2: rerank or filter.** Compute stronger relevance or quality scores and keep a refined set  $\hat{z}_{1:k}$ .

**Step 3: compress or structure.** If the evidence is too large or the question is corpus-global, map it to a compressed or structured representation  $c(\hat{z}_{1:k})$  or graph summary  $G$ .

**Step 4: generate with evidence.** Condition the generator on the selected evidence:

$$y \sim p_{\theta}(y \mid x, c(\hat{z}_{1:k})). \quad (6.12)$$

**Step 5: critique or verify.** Estimate faithfulness using a checker, reflection tokens, or consistency tests.

**Step 6: attach citations.** For each answer sentence or claim  $c_j$ , output a span  $z(c_j)$  and optionally optimize a citation reward such as Eq. (6.11).

**Step 7: train or decode with a joint objective.** A generic objective is again Eq. (6.6). Different papers vary mainly in where they intervene: retrieval, correction, generation, critique, or citation.

**Algorithmic template****Framework B: final recipe.**

1. Decide whether retrieval is needed and at what granularity (document, passage, chunk, graph community).
2. Retrieve candidates with high recall.
3. Rerank or correct for precision and robustness.
4. Compress or structure the evidence to fit the prompt budget without losing critical details.
5. Generate the answer conditioned on evidence rather than on parametric memory alone.

6. Verify answer claims explicitly when possible.

7. Attach citations and evaluate claim-level support.

6.10 How the community evaluates Framework B

**TruthfulQA** (2021/2022): measure whether models imitate falsehoods common in human text. *Why it mattered*: it made truthfulness a distinct evaluation target rather than a side effect of capability [14]..

**HaluEval** (2023): benchmark hallucination behaviors in instruction-following settings. *Why it mattered*: it provided a large-scale evaluation set for hallucination analysis [17]..

**FactScore** (2023): measure factual precision of long-form generations at an atomic-claim level. *Why it mattered*: it encouraged claim-level rather than whole-answer factual evaluation [18]..

**LongBench-Cite / LongCite** (2024/2025): evaluate long-context question answering with fine-grained citations. *Why it mattered*: it pushed citation quality into the mainstream evaluation loop [37]..

What to remember

Framework B is broader than “RAG.” It includes retrieval, reranking, compression, critique, correction, graph structuring, and citation alignment. Hallucination reduction is a control problem over the whole evidence pipeline.

7 Recent influential works (2023–2025), separated into canon and frontier

Plain-language summary

This section is intentionally selective. The goal is not to list every paper. The goal is to highlight the works that most strongly changed how researchers and practitioners think about long context and hallucination reduction. A second table then lists promising frontier papers whose long-term status is still forming.

7.1 Main-text canon: the papers a newcomer should know first

| Work    | Framework | Main improvement                               | Why it changed community practice                                |
|---------|-----------|------------------------------------------------|------------------------------------------------------------------|
| PI [19] | A         | Linear remapping of positions for RoPE models. | Became the canonical simple baseline for long-context extension. |

| Work                                   | Framework      | Main improvement                                                                  | Why it changed community practice                                         |
|----------------------------------------|----------------|-----------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| YaRN [22]                              | A              | More efficient and refined RoPE-extension recipe.                                 | Widely treated as a practical improvement over plain PI.                  |
| Lost in the Middle [20]                | A (evaluation) | Showed strong position sensitivity, especially for middle evidence.               | Changed evaluation norms: location matters, not just nominal window size. |
| Retrieval meets Long Context LLMs [43] | A+B bridge     | Compared retrieval augmentation with long-window extension and advocated hybrids. | Reframed a false dichotomy that many practitioners still face.            |
| LongLoRA [24]                          | A              | Cheap long-context adaptation recipe for open models.                             | Lowered the barrier to extending context windows in practice.             |
| LongBench [21]                         | A (evaluation) | First broad multilingual multi-task benchmark.                                    | Gave the field a shared long-context evaluation platform.                 |
| Self-RAG [32]                          | B              | Adaptive retrieval plus self-critique.                                            | Made retrieval behavior controllable inside generation.                   |
| GraphRAG [34]                          | B              | Graph-based corpus structuring for global questions.                              | Popularized structured retrieval beyond flat passage search.              |
| LongCite [37]                          | B              | Direct training for fine-grained citation generation.                             | Moved citation from a side concern to a trainable primary target.         |
| LongRoPE [31]                          | A              | Searched nonuniform interpolation plus staged extension.                          | Marked the million-scale context-window milestone in RoPE extension.      |

## 7.2 How each canonical paper fits the two-framework view

### 7.2.1 PI

#### Layer 1: intuition

PI matters because it made long-context extension conceptually simple: shrink positions before RoPE. The community could then argue about *how* to improve that basic idea rather than about whether the whole direction was viable.

*Claim and evidence.* PI claims that long-context extension can be achieved with minimal

fine-tuning while preserving short-context quality. The paper supports this with long-context retrieval, language-modeling, and summarization experiments together with a theoretical stability argument about why interpolation is gentler than raw extrapolation [19].

#### Layer 2: core mechanism

Its final algorithm is exactly Eq. (5.4): replace raw position  $m$  by  $mL_{\text{train}}/L_{\text{test}}$  and continue training briefly on longer sequences.

### 7.2.2 YaRN

#### Layer 1: intuition

YaRN’s influence comes from being a better recipe, not from rejecting PI. It refined PI. That is why it fits naturally under Framework A rather than competing with it.

*Claim and evidence.* YaRN claims a more compute-efficient context-extension path than earlier interpolation methods. It substantiates that claim through reduced training-token and step requirements plus strong long-context benchmark results [22].

#### Layer 2: core mechanism

Its effective final algorithm is: choose a more nuanced interpolation schedule than global linear scaling, then adapt with a compute-efficient long-context training recipe [22].

### 7.2.3 Lost in the Middle

#### Layer 1: intuition

Lost in the Middle did not introduce a new training method. It did something equally important: it exposed that many supposed long-context models were really weak at using middle-position evidence.

*Claim and evidence.* Its central claim is diagnostic: position matters strongly even for models marketed as long-context capable. It supports that claim with controlled sweeps over evidence location in multi-document QA and key-value retrieval tasks [20].

#### Layer 2: core mechanism

Its experimental design varies the location of answer-bearing evidence while keeping other factors controlled. In our notation, it directly probes the stability gap  $\Delta_{\text{LC}}(f; \ell, \pi)$  as a function of  $\pi$  [20].

### 7.2.4 Retrieval meets Long Context LLMs

#### Layer 1: intuition

This paper matters because many practitioners must decide between buying a larger context window and building a retrieval system. The paper argued that this is often the wrong framing: retrieval and long context can be complementary, and retrieval can remain computationally attractive even when long windows are available.

*Claim and evidence.* The paper substantiates this with comparisons across multiple long-context tasks, showing that retrieval-augmented models can match or beat some longer-window baselines at lower cost, and that hybrids often work best [43].

#### Layer 2: core mechanism

Conceptually, the paper compares two ways of reducing long-task risk: improve inside-model context handling (Framework A) or improve evidence selection outside the model (Framework B), then shows hybrids can outperform either alone [43].

### 7.2.5 LongLoRA

#### Layer 1: intuition

LongLoRA was influential because many open-weight users cared less about an elegant theory and more about a feasible recipe that worked on limited hardware.

*Claim and evidence.* LongLoRA claims that sparse training-time attention and parameter-efficient updates are enough to extend strong open models to much longer contexts. It backs this with context-extension results on Llama-family models under limited compute budgets [24].

#### Layer 2: core mechanism

Its key derivational move is to keep the long-context objective but approximate training-time attention more cheaply while only updating a small parameter subset plus selected non-LoRA components [24].

### 7.2.6 Self-RAG

#### Layer 1: intuition

Self-RAG matters because it turned retrieval into something the model can invoke or not invoke depending on need. That is a conceptual shift from static RAG pipelines.

*Claim and evidence.* Self-RAG claims gains in answer quality, factuality, and citation behavior by combining retrieval with self-reflection. It substantiates those claims on open-domain QA, reasoning, fact verification, and long-form generation settings [32].

**Layer 2: core mechanism**

Its final algorithm couples generation with reflection tokens and retrieval-control tokens, effectively learning a retrieval-conditioned policy over answer production [32].

**7.2.7 GraphRAG****Layer 1: intuition**

GraphRAG became influential because many real enterprise questions are not local fact lookups. They are corpus-level synthesis questions. A graph and its community summaries are a natural intermediate representation for such tasks.

*Claim and evidence.* The paper claims that graph-based community summaries outperform naive passage RAG for global sensemaking questions. It supports that claim with evaluations of answer comprehensiveness and diversity on large private corpora [34].

**Layer 2: core mechanism**

Its final algorithm is a two-stage representation learning pipeline: build an entity graph and community summaries offline, then answer new queries by combining community-level partial responses [34].

**7.2.8 LongCite****Layer 1: intuition**

LongCite matters because it made citation generation something models can be trained to do well, not something we merely hope emerges from good answering.

*Claim and evidence.* LongCite claims that direct supervision for long-context QA with citations substantially improves sentence-level attribution quality. It substantiates that claim using its LongBench-Cite benchmark and strong comparisons to existing open and proprietary models [37].

**Layer 2: core mechanism**

Its final algorithm uses automatically constructed long-context QA-with-citations data and trains the model to output answer text paired with fine-grained sentence-level citations [37].

**7.2.9 LongRoPE****Layer 1: intuition**

LongRoPE's influence comes from scale. It showed that million-scale windows are reachable without full pretraining from scratch if the remapping and adaptation schedule are carefully engineered.

*Claim and evidence.* LongRoPE claims that searched nonuniform interpolation plus staged

adaptation can reach extremely long windows while keeping short-context quality usable. It backs that claim with million-scale context experiments and multi-benchmark long-context evaluations [31].

### Layer 2: core mechanism

Its final algorithm combines searched nonuniform remapping with staged continued training and short-context readjustment [31].

## 7.3 Promising frontier papers whose final status is still maturing

| Work             | Framework | Why it is interesting but still maturing                                                                            |
|------------------|-----------|---------------------------------------------------------------------------------------------------------------------|
| PoSE [23]        | A         | Clever positional training distribution shift; strong recipe value, but narrower conceptual influence than PI/YaRN. |
| LongAlign [30]   | A         | Important alignment recipe for long instruction data; impact is practical more than foundational.                   |
| ReAttention [39] | A         | Attractive training-free infinite-context idea; broad long-term adoption remains to be seen.                        |
| LongRoPE2 [40]   | A         | Strong near-lossless scaling claims with mixed-window preservation; too recent to call fully canonical.             |
| UltraLong [41]   | A         | Important large-scale training recipe for 1M–4M contexts; again very recent.                                        |
| CRAG [33]        | B         | Strong corrective retrieval idea, especially when retrieval quality is unreliable.                                  |
| SelfCite [42]    | B         | Elegant self-supervised citation reward; clearly promising, but too recent to rank as settled canon.                |

## 7.4 A practical lesson from the recent literature

**What to remember**

The recent literature converges on a stable story.

1. **For long-context stability**, simple position remapping works surprisingly well, but stronger results usually require careful adaptation recipes and evaluation beyond passkey tests.
2. **For hallucination reduction**, retrieval alone is not enough. Systems increasingly add critique, correction, structure, and citation supervision.
3. **For real applications**, hybrid systems are usually strongest: moderately long context plus targeted retrieval plus verification/citation.

## 8 Design rules, failure-analysis checklist, and open problems

### 8.1 When should you use a larger context window, retrieval, or both?

| Situation                                                 | Best default                                               | Why                                                                                    |
|-----------------------------------------------------------|------------------------------------------------------------|----------------------------------------------------------------------------------------|
| Single long document; answer depends on many nearby parts | Larger context window                                      | The evidence is already local to one source; retrieval may add unnecessary complexity. |
| Huge corpus; answer depends on a few tiny passages        | Retrieval or hybrid                                        | Reading the whole corpus is wasteful; targeted evidence selection is more efficient.   |
| Question needs corpus-wide synthesis                      | GraphRAG-style structure or hybrid                         | Flat retrieval often misses global themes and relationships.                           |
| Regulated or high-stakes setting requiring auditability   | Hybrid with citation                                       | Verifiability matters as much as answer quality.                                       |
| Open-weight deployment on limited hardware                | Recipe-based long-context adaptation + selective retrieval | Cost matters; full long-context pretraining may be infeasible.                         |

**Table 7:** A practical decision table.

### 8.2 A failure-analysis checklist for practitioners

**Algorithmic template**

When a long-context or RAG system fails, diagnose in this order.

1. **Was the right evidence present in the raw corpus?**
2. **Did retrieval return it at all?**
3. **Did reranking or compression discard it?**
4. **If the evidence was in the final prompt, where was it positioned?**
5. **Did the model answer from the evidence, or from unsupported parametric memory?**

### 6. Was the answer right but the citation wrong?

This checklist prevents the common mistake of blaming the generator for an upstream evidence-selection error.

## 8.3 What remains genuinely hard

There are still several open problems.

1. **True million-scale reasoning.** Reading a million tokens is easier than reasoning correctly over them.
2. **Exception preservation under compression.** Small exception clauses are easy to lose when summarizing evidence.
3. **Reliable claim decomposition.** Citation and factual evaluation both depend on decomposing answers into meaningful atomic claims.
4. **Joint optimization of short- and long-context quality.** Many extension methods still trade one regime against the other.
5. **Robust evaluation.** Synthetic tests remain useful, but they cannot stand in for realistic long-context reasoning and attribution tasks.

## 8.4 If you only remember six ideas from the whole survey

### Plain-language summary

1. A context window is not the same as long-context skill.
2. Most modern long-context extension recipes can be understood as position remapping plus careful adaptation.
3. Retrieval and long context are complementary, not mutually exclusive.
4. Hallucination reduction is a pipeline problem: retrieve, rerank, compress, generate, verify, cite.
5. Citation is a stronger requirement than answer correctness because it makes verification fast.
6. The strongest systems usually combine Framework A and Framework B rather than committing to only one of them.

## 9 Conclusion

This note started with tokens, next-token prediction, attention, position, retrieval, and support. It then grouped the historical roots by the bottleneck each paper targeted: dense attention, memory extension, cost reduction, position geometry, and external evidence. From there, it showed that much of the modern literature can be understood through two common frameworks.

**Framework A** treats long-context extension as a geometry-preservation problem for attention under large positions. This explains why PI, YaRN, PoSE, LongLoRA, LongAlign, LongRoPE, and LongRoPE2 are related rather than isolated tricks. **Framework B** treats hallucination reduction as evidence-grounded selective generation. This explains why retrieval, reranking, correction, graph structuring, verification, and citation methods all belong to one broader family.

The field’s practical lesson is now clear. A large context window alone does not guarantee good use of context. Retrieval alone does not guarantee faithful answers. The most dependable systems stabilize long-context behavior *inside the model* and control evidence *around the model*. That is the real synthesis from root to frontier.

## A Optional reading path for an even slower first pass

If you are still new to the topic, reread only the following in order.

1. Section 2 on tokens, next-token prediction, attention, retrieval, hallucination, and citation.
2. The Transformer, Transformer-XL, ALiBi, RoPE, and RAG subsections in Section 4.
3. PI in Section 5 and Self-RAG in Section 6.
4. The canon table in Section 7.

Only after that should you study LongRoPE, GraphRAG, LongCite, and SelfCite in detail.

## B Notation summary

| Symbol                      | Meaning                                                        |
|-----------------------------|----------------------------------------------------------------|
| $w_t$                       | token ID at position $t$                                       |
| $x_t$                       | embedding vector of token $w_t$                                |
| $H$                         | matrix of hidden states across positions                       |
| $Q, K, V$                   | query, key, and value matrices                                 |
| $M$                         | causal or sparse attention mask                                |
| $\ell$                      | input length in tokens                                         |
| $\pi$                       | position of answer-bearing evidence inside the input           |
| $\mathcal{M}(f; \ell, \pi)$ | task metric as a function of length and evidence position      |
| $\text{supp}(c, E)$         | indicator that claim $c$ is supported by evidence $E$          |
| $z$                         | latent evidence variable (passage, chunk, graph summary, etc.) |
| $g(m)$                      | position remapping function for long-context extension         |
| $\mathcal{B}$               | prompt-budget constraint for evidence packing                  |

## References

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention Is All You Need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [2] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context. In *ACL*, 2019.
- [3] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, Chloe Hillier, and Timothy P. Lillicrap. Compressive Transformers for Long-Range Sequence Modelling. In *ICLR*, 2020.
- [4] Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The Efficient Transformer. In *ICLR*, 2020.
- [5] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The Long-Document Transformer. arXiv:2004.05150, 2020.
- [6] Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontañón, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big Bird: Transformers for Longer Sequences. In *NeurIPS*, 2020.
- [7] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. REALM: Retrieval-Augmented Language Model Pre-Training. In *ICML*, 2020.
- [8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *NeurIPS*, 2020.
- [9] Gautier Izacard and Edouard Grave. Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering. In *EACL*, 2021.
- [10] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced Transformer with Rotary Position Embedding. arXiv:2104.09864, 2021.
- [11] Ofir Press, Noah A. Smith, and Mike Lewis. Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation. In *ICLR*, 2022.
- [12] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, Diego de las Casas, et al. Improving Language Models by Retrieving from Trillions of Tokens. In *ICML*, 2022.
- [13] Yuhuai Wu, Markus N. Rabe, Wenda Li, Jimmy Ba, Roger Grosse, and Christian Szegedy. Memorizing Transformers. In *ICLR*, 2022.
- [14] Stephanie Lin, Jacob Hilton, and Owain Evans. TruthfulQA: Measuring How Models Mimic Human Falsehoods. In *ACL*, 2022.

- 
- [15] Luyu Gao, Zhuyun Dai, Panupong Pasupat, Anthony Chen, Arun Tejasvi Chaganty, Yicheng Fan, Vincent Y. Zhao, Ni Lao, Hongrae Lee, Da-Cheng Juan, and others. RARR: Researching and Revising What Language Models Say, Using Language Models. In *ACL*, 2023.
  - [16] Potsawee Manakul, Adian Liusie, and Mark J. F. Gales. SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. In *EMNLP*, 2023.
  - [17] Junyi Li, Xiaoxue Cheng, Wayne Xin Zhao, Jian-Yun Nie, and Ji-Rong Wen. HaluEval: A Large-Scale Hallucination Evaluation Benchmark for Large Language Models. In *EMNLP*, 2023.
  - [18] Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis, Wen-tau Yih, Pang Wei Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. In *EMNLP*, 2023.
  - [19] Shouyuan Chen, Sherman Wong, Liangjian Chen, and Yuandong Tian. Extending Context Window of Large Language Models via Positional Interpolation. arXiv:2306.15595, 2023.
  - [20] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*, 2024. Preprint 2023.
  - [21] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. arXiv:2308.14508, 2023.
  - [22] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. YaRN: Efficient Context Window Extension of Large Language Models. arXiv:2309.00071, 2023.
  - [23] Dawei Zhu, Nan Yang, Liang Wang, Yifan Song, Wenhao Wu, Furu Wei, and Sujian Li. PoSE: Efficient Context Window Extension of LLMs via Positional Skip-wise Training. arXiv:2309.10400, 2023.
  - [24] Yukang Chen, Shengju Qian, Haotian Tang, Xin Lai, Zhijian Liu, Song Han, and Jiaya Jia. LongLoRA: Efficient Fine-tuning of Long-Context Large Language Models. arXiv:2309.12307, 2023.
  - [25] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Zhenyu Zhang, Song Han, and Mike Lewis. Efficient Streaming Language Models with Attention Sinks. In *ICLR*, 2024. Preprint 2023.
  - [26] Shehzaad Dhuliawala, Mojtaba Komeili, Jing Xu, Roberta Raileanu, Xian Li, Asli Celikyilmaz, and Jason Weston. Chain-of-Verification Reduces Hallucination in Large Language Models. In *Findings of ACL*, 2024. Preprint 2023.
  - [27] Xinrong Zhang, Yingfa Chen, Shengding Hu, Zihang Xu, Junhao Chen, Moo Khai Hao, Xu Han, Zhen Leng Thai, Shuo Wang, Zhiyuan Liu, and Maosong Sun.  $\infty$ Bench: Extending Long Context Evaluation Beyond 100K Tokens. arXiv:2402.13718, 2024.

- 
- [28] Cheng-Ping Hsieh, Simeng Sun, Samuel Krizan, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. RULER: What’s the Real Context Size of Your Long-Context Language Models? arXiv:2404.06654, 2024.
  - [29] Tsendsuren Munkhdalai, Manaal Faruqui, and Siddharth Gopal. Leave No Context Behind: Efficient Infinite Context Transformers with Infini-attention. arXiv:2404.07143, 2024.
  - [30] Yushi Bai, Xin Lv, Jiajie Zhang, Yuze He, Ji Qi, Lei Hou, Jie Tang, Yuxiao Dong, and Juanzi Li. LongAlign: A Recipe for Long Context Alignment of Large Language Models. arXiv:2401.18058, 2024.
  - [31] Yiran Ding, Li Lyna Zhang, Chengruidong Zhang, Yuanyuan Xu, Ning Shang, Jiahang Xu, Fan Yang, and Mao Yang. LongRoPE: Extending LLM Context Window Beyond 2 Million Tokens. arXiv:2402.13753, 2024.
  - [32] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. In *ICLR*, 2024. Preprint 2023.
  - [33] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. Corrective Retrieval Augmented Generation. arXiv:2401.15884, 2024.
  - [34] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv:2404.16130, 2024.
  - [35] Hui Jiang, Xinyu Ma, and Weizhu Chen. LongRAG: Enhancing Retrieval-Augmented Generation with Long-context LLMs. arXiv:2406.15319, 2024.
  - [36] Feng Ye, Shuangyin Li, Yibo Chen, and others. R<sup>2</sup>AG: Incorporating Retrieval Information into Retrieval-Augmented Generation. arXiv:2406.13249, 2024.
  - [37] Jiajie Zhang, Yushi Bai, Xin Lv, Wanjuan Gu, Danqing Liu, Minhao Zou, Shulin Cao, Lei Hou, Yuxiao Dong, Ling Feng, and Juanzi Li. LongCite: Enabling LLMs to Generate Fine-grained Citations in Long-context QA. arXiv:2409.02897, 2024.
  - [38] Yushi Bai, Shangqing Tu, Jiajie Zhang, Hao Peng, Xiaozhi Wang, Xin Lv, Shulin Cao, Jiazheng Xu, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. LongBench v2: Towards Deeper Understanding and Reasoning on Realistic Long-context Multitasks. arXiv:2412.15204, 2024.
  - [39] Xiaoran Liu, Ruixiao Li, Qipeng Guo, Zhigeng Liu, Yuerong Song, Kai Lv, Hang Yan, Linlin Li, Qun Liu, and Xipeng Qiu. ReAttention: Training-Free Infinite Context with Finite Attention Scope. arXiv:2407.15176, 2024/2025.
  - [40] Ning Shang, Li Lyna Zhang, Siyuan Wang, Gaokai Zhang, Gilsinia Lopez, Fan Yang, Weizhu Chen, and Mao Yang. LongRoPE2: Near-Lossless LLM Context Window Scaling. arXiv:2502.20082, 2025.
  - [41] Chejian Xu, Wei Ping, Peng Xu, Zihan Liu, Boxin Wang, Mohammad Shoeybi, Bo Li, and Bryan Catanzaro. From 128K to 4M: Efficient Training of Ultra-Long Context Large Language Models. arXiv:2504.06214, 2025.

- 
- [42] Yung-Sung Chuang, Benjamin Cohen-Wang, Shannon Zejiang Shen, Zhaofeng Wu, Hu Xu, Xi Victoria Lin, James Glass, Shang-Wen Li, and Wen-tau Yih. SelfCite: Self-Supervised Alignment for Context Attribution in Large Language Models. arXiv:2502.09604, 2025.
  - [43] Peng Xu, Wei Ping, Xianchao Wu, Lawrence McAfee, Chen Zhu, Zihan Liu, Sandeep Subramanian, Evelina Bakhturina, Mohammad Shoeybi, and Bryan Catanzaro. Retrieval Meets Long Context Large Language Models. arXiv:2310.03025, 2023.